

EXECUTION MINDSET

Think as:

- A principal engineer at a well-funded AI startup
- A systems architect who has shipped Electron + Rust + Python desktop apps
- An NLE engineer who understands DaVinci Resolve and Premiere internals
- A GPU systems programmer preparing for CUDA/Vulkan integration
- A product designer who ships cinematic, dark, modern UIs

Your code must be:

- Production-grade (no toy code)
- Strongly typed throughout (TypeScript strict mode, Rust with full type safety)
- Modular and extensible (every system must be replaceable)
- Non-destructive (timeline operations never mutate source media)
- Future-ready (stubs and interfaces for GPU, distributed inference, AI agents)

PHASE 1 — MONOREPO INITIALIZATION

1.1 Root Structure

Create the following folder structure at the workspace root:

```
axew/
├── apps/
│   ├── desktop/           # Electron + React frontend
│   └── ai-service/        # Python FastAPI AI runtime
├── crates/
│   └── axew-core/         # Rust backend engine
├── packages/
│   ├── shared-types/     # Shared TypeScript types
│   └── ui-components/    # Shared React component library
├── scripts/              # Dev/build/start scripts
├── docs/                 # Architecture documentation
├── .env.example
├── package.json           # Root workspace package.json (pnpm workspaces)
├── pnpm-workspace.yaml
├── turbo.json             # Turborepo config
└── README.md
```

1.2 Root Package Configuration

Create `package.json` at root:

json

```
{
  "name": "axew",
  "version": "0.1.0",
  "private": true,
  "scripts": {
    "dev": "turbo run dev --parallel",
    "build": "turbo run build",
    "start": "turbo run start",
    "lint": "turbo run lint",
    "type-check": "turbo run type-check",
    "clean": "turbo run clean"
  },
  "devDependencies": {
    "turbo": "latest",
    "@types/node": "^20.0.0",
    "typescript": "^5.4.0"
  },
  "packageManager": "pnpm@9.0.0",
  "engines": {
    "node": ">=20.0.0",
    "pnpm": ">=9.0.0"
  }
}
```

Create `pnpm-workspace.yaml`:

yaml

```
packages:
  - 'apps/*'
  - 'packages/*'
```

Create `turbo.json`:

json

```
{
  "$schema": "https://turbo.build/schema.json",
  "globalDependencies": [".env"],
  "pipeline": {
    "build": {
      "dependsOn": ["^build"],
      "outputs": ["dist/**", ".next/**", "build/**"]
    },
    "dev": {
      "cache": false,
      "persistent": true
    },
    "start": {
      "cache": false,
      "persistent": true
    },
    "lint": {
      "outputs": []
    },
    "type-check": {
      "outputs": []
    },
    "clean": {
      "cache": false
    }
  }
}
```

```
}  
}  
}  
  
Create .env.example:  
AXEW_RUST_PORT=7001  
AXEW_AI_PORT=7002  
AXEW_ENV=development  
AXEW_FFMPEG_PATH=ffmpeg  
AXEW_FFPROBE_PATH=ffprobe  
AXEW_MODELS_DIR=~/.axew/models  
AXEW_PROJECTS_DIR=~/.axew/projects  
AXEW_CACHE_DIR=~/.axew/cache  
OLLAMA_HOST=http://localhost:11434  
Copy .env.example to .env.
```

PHASE 1 — DESKTOP APP INITIALIZATION (Electron + React + TypeScript)

1.3 Desktop App Bootstrap

Navigate to `apps/desktop/`. Create the following `package.json`:

json

```
{  
  "name": "@axew/desktop",  
  "version": "0.1.0",  
  "private": true,  
  "main": "dist-electron/main.js",  
  "scripts": {  
    "dev": "vite",  
    "build": "tsc && vite build && electron-builder",  
    "start": "electron .",  
    "dev:electron": "concurrently \"pnpm dev\" \"wait-on http://localhost:5173 && electron .\"",  
    "type-check": "tsc --noEmit",  
    "lint": "eslint src --ext .ts,.tsx"  
  },  
  "dependencies": {  
    "react": "^18.3.0",  
    "react-dom": "^18.3.0",  
    "zustand": "^4.5.0",  
    "framer-motion": "^11.0.0",  
    "@radix-ui/react-context-menu": "^2.1.5",  
    "@radix-ui/react-dialog": "^1.0.5",  
    "@radix-ui/react-dropdown-menu": "^2.0.6",  
    "@radix-ui/react-scroll-area": "^1.0.5",  
    "@radix-ui/react-slider": "^1.1.2",  
    "@radix-ui/react-tooltip": "^1.0.7",  
    "@radix-ui/react-tabs": "^1.0.4",  
    "@radix-ui/react-separator": "^1.0.3",  
    "lucide-react": "^0.378.0",  
    "clsx": "^2.1.0",  
    "tailwind-merge": "^2.3.0",  
    "axios": "^1.6.0",  
    "react-rnd": "^10.4.1",
```

```

    "immer": "^10.1.1",
    "@tanstack/react-query": "^5.35.0",
    "react-virtualized-auto-sizer": "^1.0.22",
    "react-window": "^1.8.10",
    "date-fns": "^3.6.0"
  },
  "devDependencies": {
    "@types/react": "^18.3.0",
    "@types/react-dom": "^18.3.0",
    "@types/react-window": "^1.8.8",
    "@vitejs/plugin-react": "^4.2.0",
    "autoprefixer": "^10.4.19",
    "concurrently": "^8.2.2",
    "electron": "^30.0.0",
    "electron-builder": "^24.6.0",
    "eslint": "^8.57.0",
    "postcss": "^8.4.38",
    "tailwindcss": "^3.4.3",
    "typescript": "^5.4.0",
    "vite": "^5.2.0",
    "vite-plugin-electron": "^0.28.0",
    "vite-plugin-electron-renderer": "^0.14.0",
    "wait-on": "^7.2.0"
  },
  "build": {
    "appId": "com.axew.desktop",
    "productName": "AXEW",
    "directories": {
      "output": "release"
    },
    "files": ["dist/**", "dist-electron/**"],
    "mac": {
      "category": "public.app-category.video",
      "target": "dmg"
    },
    "win": {
      "target": "nsis"
    },
    "linux": {
      "target": "AppImage"
    }
  }
}

```

Install dependencies by running: `cd apps/desktop && pnpm install`

1.4 Vite Configuration

Create `apps/desktop/vite.config.ts`:

typescript

```

import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import electron from 'vite-plugin-electron'
import renderer from 'vite-plugin-electron-renderer'
import path from 'path'

export default defineConfig({
  plugins: [

```

```

    react(),
    electron([
      {
        entry: 'electron/main.ts',
        vite: {
          build: {
            outDir: 'dist-electron',
            rollupOptions: {
              external: ['electron']
            }
          }
        }
      },
      {
        entry: 'electron/preload.ts',
        onStart(options) {
          options.reload()
        },
        vite: {
          build: {
            outDir: 'dist-electron',
            rollupOptions: {
              external: ['electron']
            }
          }
        }
      }
    ]),
    renderer()
  ],
  resolve: {
    alias: {
      '@': path.resolve(__dirname, './src'),
      '@shared': path.resolve(__dirname, '../..../packages/shared-types/src')
    }
  },
  css: {
    postcss: './postcss.config.js'
  }
}))

```

1.5 TypeScript Configuration

Create `apps/desktop/tsconfig.json`:

json

```

{
  "compilerOptions": {
    "target": "ES2022",
    "useDefineForClassFields": true,
    "lib": ["ES2022", "DOM", "DOM.Iterable"],
    "module": "ESNext",
    "skipLibCheck": true,
    "moduleResolution": "bundler",
    "allowImportingTsExtensions": true,
    "resolveJsonModule": true,
    "isolatedModules": true,
    "noEmit": true,

```

```

    "jsx": "react-jsx",
    "strict": true,
    "noUnusedLocals": true,
    "noUnusedParameters": true,
    "noFallthroughCasesInSwitch": true,
    "baseUrl": ".",
    "paths": {
      "@/*": ["src/*"],
      "@shared/*": ["../../packages/shared-types/src/*"]
    }
  },
  "include": ["src", "electron"],
  "exclude": ["node_modules", "dist", "dist-electron"]
}

```

1.6 TailwindCSS Configuration

Create `apps/desktop/tailwind.config.js`:

javascript

```

/** @type {import('tailwindcss').Config} */
export default {
  content: [
    "./index.html",
    "./src/**/*..{js,ts,jsx,tsx}",
  ],
  theme: {
    extend: {
      colors: {
        axew: {
          bg: '#0A0A0C',
          surface: '#111115',
          panel: '#16161B',
          border: '#22222A',
          borderLight: '#2E2E3A',
          accent: '#5B5BFF',
          accentHover: '#7070FF',
          accentSubtle: '#1A1A3E',
          text: '#E8E8F0',
          textMuted: '#6B6B7E',
          textDim: '#3E3E4E',
          timeline: '#0D0D10',
          playhead: '#FF4444',
          waveform: '#3D6B8A',
          clip: '#1E3A5F',
          clipBorder: '#2A5080',
          clipHover: '#254570',
          success: '#22C55E',
          warning: '#F59E0B',
          error: '#EF4444',
          ai: '#A855F7',
          aiGlow: '#7C3AED',
        }
      },
      fontFamily: {
        sans: ['Inter', 'system-ui', 'sans-serif'],
        mono: ['JetBrains Mono', 'Fira Code', 'monospace'],
        display: ['Inter', 'system-ui', 'sans-serif'],
      }
    }
  }
}

```

```

    },
    fontSize: {
      '2xs': ['10px', '14px'],
      xs: ['11px', '16px'],
      sm: ['12px', '18px'],
      base: ['13px', '20px'],
      md: ['14px', '20px'],
      lg: ['16px', '24px'],
    },
    borderRadius: {
      DEFAULT: '4px',
      sm: '2px',
      md: '6px',
      lg: '10px',
      xl: '14px',
    },
    boxShadow: {
      panel: '0 4px 24px rgba(0,0,0,0.6)',
      glow: '0 0 20px rgba(91,91,255,0.3)',
      aiGlow: '0 0 20px rgba(168,85,247,0.4)',
      clip: '0 2px 8px rgba(0,0,0,0.5)',
    },
    animation: {
      'pulse-slow': 'pulse 3s ease-in-out infinite',
      'fade-in': 'fadeIn 0.2s ease-out',
      'slide-up': 'slideUp 0.2s ease-out',
      'shimmer': 'shimmer 1.5s infinite',
    },
    keyframes: {
      fadeIn: {
        '0%': { opacity: '0' },
        '100%': { opacity: '1' },
      },
      slideUp: {
        '0%': { transform: 'translateY(4px)', opacity: '0' },
        '100%': { transform: 'translateY(0)', opacity: '1' },
      },
      shimmer: {
        '0%': { backgroundPosition: '-200% 0' },
        '100%': { backgroundPosition: '200% 0' },
      },
    },
  },
  plugins: [],
}

```

Create `apps/desktop/postcss.config.js`:

javascript

```

export default {
  plugins: [
    tailwindcss: {},
    autoprefixer: {},
  ],
}

```

PHASE 1 — ELECTRON MAIN PROCESS

Create `apps/desktop/electron/main.ts`:

typescript

```
import { app, BrowserWindow, ipcMain, dialog, shell, Menu } from 'electron'
import path from 'path'
import { spawn, ChildProcess } from 'child_process'
import fs from 'fs'

const isDev = process.env.NODE_ENV === 'development' || !app.isPackaged

let mainWindow: BrowserWindow | null = null
let rustProcess: ChildProcess | null = null
let aiProcess: ChildProcess | null = null

const RUST_PORT = process.env.AXEW_RUST_PORT || '7001'
const AI_PORT = process.env.AXEW_AI_PORT || '7002'

function createWindow(): void {
  mainWindow = new BrowserWindow({
    width: 1600,
    height: 1000,
    minWidth: 1200,
    minHeight: 700,
    backgroundColor: '#0A0A0C',
    titleBarStyle: process.platform === 'darwin' ? 'hidden' : 'default',
    titleBarOverlay: process.platform !== 'darwin' ? {
      color: '#0A0A0C',
      symbolColor: '#E8E8F0',
      height: 36
    } : undefined,
    trafficLightPosition: { x: 16, y: 10 },
    webPreferences: {
      preload: path.join(__dirname, 'preload.js'),
      contextIsolation: true,
      nodeIntegration: false,
      webSecurity: true,
    },
    frame: process.platform !== 'darwin',
    show: false,
  })

  mainWindow.once('ready-to-show', () => {
    mainWindow?.show()
    if (isDev) {
      mainWindow?.webContents.openDevTools({ mode: 'detach' })
    }
  })

  if (isDev) {
    mainWindow.loadURL('http://localhost:5173')
  } else {
    mainWindow.loadFile(path.join(__dirname, '../dist/index.html'))
  }

  mainWindow.on('closed', () => {
    mainWindow = null
  })

  setupMenu()
}
```

```

    ]
}

function setupMenu(): void {
    const template: Electron.MenuItemConstructorOptions[] = [
        {
            label: 'AXEW',
            submenu: [
                { label: 'About AXEW', role: 'about' },
                { type: 'separator' },
                { label: 'Quit', accelerator: 'CmdOrCtrl+Q', click: () =>
app.quit() }
            ]
        },
        {
            label: 'File',
            submenu: [
                { label: 'New Project', accelerator: 'CmdOrCtrl+N', click: () =>
mainWindow?.webContents.send('menu:new-project') },
                { label: 'Open Project', accelerator: 'CmdOrCtrl+O', click: () =>
mainWindow?.webContents.send('menu:open-project') },
                { label: 'Save Project', accelerator: 'CmdOrCtrl+S', click: () =>
mainWindow?.webContents.send('menu:save-project') },
                { type: 'separator' },
                { label: 'Import Media', accelerator: 'CmdOrCtrl+I', click: () =>
mainWindow?.webContents.send('menu:import-media') },
                { type: 'separator' },
                { label: 'Export', accelerator: 'CmdOrCtrl+E', click: () =>
mainWindow?.webContents.send('menu:export') },
            ]
        },
        {
            label: 'Edit',
            submenu: [
                { role: 'undo' },
                { role: 'redo' },
                { type: 'separator' },
                { role: 'cut' },
                { role: 'copy' },
                { role: 'paste' },
            ]
        },
        {
            label: 'View',
            submenu: [
                { role: 'reload' },
                { role: 'toggleDevTools' },
                { type: 'separator' },
                { role: 'togglefullscreen' }
            ]
        }
    ]

    const menu = Menu.buildFromTemplate(template)
    Menu.setApplicationMenu(menu)
}

function startRustService(): void {
    const rustBin = isDev

```

```

    ? path.join( __dirname, '../../../crates/axew-core/target/debug/axew-
core')
    : path.join(process.resourcesPath, 'axew-core')

    if (!fs.existsSync(rustBin)) {
        console.warn('[Main] Rust binary not found, skipping service start')
        return
    }

    rustProcess = spawn(rustBin, [], {
        env: { ...process.env, AXEW_PORT: RUST_PORT },
        stdio: 'pipe'
    })

    rustProcess.stdout?.on('data', (data) => {
        console.log('[Rust]', data.toString())
    })

    rustProcess.stderr?.on('data', (data) => {
        console.error('[Rust Error]', data.toString())
    })

    rustProcess.on('exit', (code) => {
        console.log(`[Rust] Process exited with code ${code}`)
    })
}

function startAIService(): void {
    const aiDir = isDev
        ? path.join(__dirname, '../../../apps/ai-service')
        : path.join(process.resourcesPath, 'ai-service')

    if (!fs.existsSync(aiDir)) {
        console.warn('[Main] AI service directory not found, skipping')
        return
    }

    const pythonBin = process.platform === 'win32' ? 'python' : 'python3'

    aiProcess = spawn(pythonBin, ['-m', 'uvicorn', 'main:app', '--host',
'127.0.0.1', '--port', AI_PORT], {
        cwd: aiDir,
        env: { ...process.env },
        stdio: 'pipe'
    })

    aiProcess.stdout?.on('data', (data) => {
        console.log('[AI Service]', data.toString())
    })

    aiProcess.stderr?.on('data', (data) => {
        console.error('[AI Service Error]', data.toString())
    })

    aiProcess.on('exit', (code) => {
        console.log(`[AI Service] Process exited with code ${code}`)
    })
}

```

```

// IPC Handlers
ipcMain.handle('dialog:openFile', async (_, options) => {
  const result = await dialog.showOpenDialog(mainWindow!, options)
  return result
})

ipcMain.handle('dialog:saveFile', async (_, options) => {
  const result = await dialog.showSaveDialog(mainWindow!, options)
  return result
})

ipcMain.handle('shell:openExternal', async (_, url: string) => {
  await shell.openExternal(url)
})

ipcMain.handle('app:getVersion', () => {
  return app.getVersion()
})

ipcMain.handle('app:getPaths', () => {
  return {
    userData: app.getPath('userData'),
    documents: app.getPath('documents'),
    downloads: app.getPath('downloads'),
    home: app.getPath('home'),
    temp: app.getPath('temp'),
  }
})

ipcMain.handle('fs:readFile', async (_, filePath: string) => {
  try {
    const content = fs.readFileSync(filePath)
    return { success: true, data: content.toString('base64') }
  } catch (err) {
    return { success: false, error: String(err) }
  }
})

ipcMain.handle('fs:writeFile', async (_, filePath: string, content: string)
=> {
  try {
    fs.mkdirSync(path.dirname(filePath), { recursive: true })
    fs.writeFileSync(filePath, content)
    return { success: true }
  } catch (err) {
    return { success: false, error: String(err) }
  }
})

ipcMain.handle('fs:exists', async (_, filePath: string) => {
  return fs.existsSync(filePath)
})

ipcMain.handle('services:getStatus', () => {
  return {
    rust: { running: rustProcess !== null && !rustProcess.killed, port:
RUST_PORT },
    ai: { running: aiProcess !== null && !aiProcess.killed, port: AI_PORT
},
  },
})

```

```

    }
  })

  app.whenReady().then(() => {
    startRustService()
    startAIService()
    createWindow()

    app.on('activate', () => {
      if (BrowserWindow.getAllWindows().length === 0) {
        createWindow()
      }
    })
  })
})

app.on('window-all-closed', () => {
  rustProcess?.kill()
  aiProcess?.kill()
  if (process.platform !== 'darwin') {
    app.quit()
  }
})

app.on('before-quit', () => {
  rustProcess?.kill()
  aiProcess?.kill()
})

```

Create `apps/desktop/electron/preload.ts`:

typescript

```

import { contextBridge, ipcRenderer } from 'electron'

type IpcListener = (...args: unknown[]) => void

const api = {
  dialog: {
    openFile: (options: Electron.OpenDialogOptions) =>
      ipcRenderer.invoke('dialog:openFile', options),
    saveFile: (options: Electron.SaveDialogOptions) =>
      ipcRenderer.invoke('dialog:saveFile', options),
  },
  shell: {
    openExternal: (url: string) =>
      ipcRenderer.invoke('shell:openExternal', url),
  },
  app: {
    getVersion: () => ipcRenderer.invoke('app:getVersion'),
    getPaths: () => ipcRenderer.invoke('app:getPaths'),
  },
  fs: {
    readFile: (filePath: string) =>
      ipcRenderer.invoke('fs:readFile', filePath),
    writeFile: (filePath: string, content: string) =>
      ipcRenderer.invoke('fs:writeFile', filePath, content),
    exists: (filePath: string) =>
      ipcRenderer.invoke('fs:exists', filePath),
  },
  services: {
    getStatus: () => ipcRenderer.invoke('services:getStatus'),
  }
}

```

```

    },
    menu: {
      on: (event: string, listener: IpcListener) => {
        ipcRenderer.on(`menu:${event}`, listener)
      },
      off: (event: string, listener: IpcListener) => {
        ipcRenderer.off(`menu:${event}`, listener)
      },
    },
  },
  ipc: {
    send: (channel: string, ...args: unknown[]) =>
      ipcRenderer.send(channel, ...args),
    invoke: (channel: string, ...args: unknown[]) =>
      ipcRenderer.invoke(channel, ...args),
    on: (channel: string, listener: IpcListener) => {
      ipcRenderer.on(channel, listener)
      return () => ipcRenderer.off(channel, listener)
    },
  },
}

```

```
contextBridge.exposeInMainWorld('axew', api)
```

```
export type AxewAPI = typeof api
```

Create `apps/desktop/electron/electron-env.d.ts`:

typescript

```
import type { AxewAPI } from './preload'
```

```

declare global {
  interface Window {
    axew: AxewAPI
  }
}

```

PHASE 1 — SHARED TYPES PACKAGE

Create `packages/shared-types/package.json`:

json

```

{
  "name": "@axew/shared-types",
  "version": "0.1.0",
  "main": "src/index.ts",
  "types": "src/index.ts",
  "exports": {
    ".": "./src/index.ts"
  }
}

```

Create `packages/shared-types/src/index.ts`:

typescript

```

export * from './timeline'
export * from './media'
export * from './project'
export * from './ai'
export * from './export'

```

Create `packages/shared-types/src/media.ts`:

typescript

```
export interface MediaFile {
  id: string
  name: string
  path: string
  type: MediaType
  duration: number // seconds
  width: number
  height: number
  fps: number
  codec: string
  audioCodec: string | null
  sampleRate: number | null
  channels: number | null
  bitrate: number // kbps
  fileSize: number // bytes
  thumbnail: string | null // base64 or file path
  createdAt: number
  updatedAt: number
  metadata: MediaMetadata
}

export type MediaType = 'video' | 'audio' | 'image' | 'sequence'

export interface MediaMetadata {
  [key: string]: string | number | boolean | null
}

export interface MediaBinFolder {
  id: string
  name: string
  parentId: string | null
  children: string[] // folder ids
  mediaIds: string[]
}
```

Create `packages/shared-types/src/timeline.ts`:

typescript

```
export interface Timeline {
  id: string
  projectId: string
  name: string
  duration: number // seconds
  frameRate: number
  width: number
  height: number
  sampleRate: number
  tracks: Track[]
  markers: Marker[]
  createdAt: number
  updatedAt: number
}

export interface Track {
  id: string
  timelineId: string
  name: string
}
```

```

    type: TrackType
    index: number
    height: number
    locked: boolean
    muted: boolean
    visible: boolean
    solo: boolean
    color: string
    clips: Clip[]
  }

export type TrackType = 'video' | 'audio' | 'subtitle' | 'fx'

export interface Clip {
  id: string
  trackId: string
  mediaId: string
  name: string
  startTime: number // timeline position in seconds
  duration: number // duration on timeline in seconds
  mediaInPoint: number // source media in point in seconds
  mediaOutPoint: number // source media out point in seconds
  speed: number // playback speed multiplier (1.0 = normal)
  opacity: number // 0-1
  volume: number // 0-2
  disabled: boolean
  color: string | null
  effects: ClipEffect[]
  transitions: {
    in: Transition | null
    out: Transition | null
  }
  keyframes: Keyframe[]
}

export interface ClipEffect {
  id: string
  type: string
  enabled: boolean
  params: Record<string, unknown>
}

export interface Transition {
  type: string
  duration: number
  params: Record<string, unknown>
}

export interface Keyframe {
  id: string
  property: string
  time: number
  value: unknown
  easing: 'linear' | 'ease-in' | 'ease-out' | 'ease-in-out' | 'bezier'
  bezierHandles?: [number, number, number, number]
}

export interface Marker {
  id: string

```

```

    time: number
    duration: number
    name: string
    color: string
    type: 'comment' | 'chapter' | 'sync' | 'ai'
    notes: string
  }

```

```

export interface PlayheadState {
  currentTime: number
  playing: boolean
  loop: boolean
  loopIn: number | null
  loopOut: number | null
  speed: number
}

```

Create `packages/shared-types/src/project.ts`:

typescript

```

import type { Timeline } from './timeline'
import type { MediaFile, MediaBinFolder } from './media'

export interface Project {
  id: string
  name: string
  description: string
  path: string
  timeline: Timeline
  mediaFiles: Record<string, MediaFile>
  mediaBin: {
    rootFolder: MediaBinFolder
    folders: Record<string, MediaBinFolder>
  }
  settings: ProjectSettings
  createdAt: number
  updatedAt: number
  version: string
}

export interface ProjectSettings {
  timeline: {
    frameRate: number
    width: number
    height: number
    sampleRate: number
  }
  playback: {
    proxyEnabled: boolean
    proxyResolution: number
    cacheSize: number
  }
  ai: {
    preferredModel: string
    autoTranscribe: boolean
    silenceThresholdDb: number
    sceneDetectionSensitivity: number
  }
  export: {
    defaultPreset: string

```

```
    outputDirectory: string
  }
}
```

```
export interface RecentProject {
  id: string
  name: string
  path: string
  thumbnail: string | null
  lastOpenedAt: number
}
```

Create `packages/shared-types/src/ai.ts`:

typescript

```
export interface AIModel {
  id: string
  name: string
  type: AIModelType
  provider: AIProvider
  size: number // GB
  quantization: string
  capabilities: AICapability[]
  status: AIModelStatus
  localPath: string | null
}

export type AIModelType = 'llm' | 'whisper' | 'embeddings' | 'vision' |
'audio'
export type AIProvider = 'ollama' | 'llama-cpp' | 'onnx' | 'pytorch' |
'openai-compatible'
export type AIModelStatus = 'available' | 'downloading' | 'loaded' |
'error' | 'not-installed'
export type AICapability = 'text' | 'vision' | 'audio' | 'code' | 'editing'

export interface AIMessage {
  id: string
  role: 'user' | 'assistant' | 'system' | 'tool'
  content: string
  timestamp: number
  model?: string
  toolCalls?: AIToolCall[]
  toolResults?: AIToolResult[]
}

export interface AIToolCall {
  id: string
  name: string
  arguments: Record<string, unknown>
}

export interface AIToolResult {
  toolCallId: string
  result: unknown
  error?: string
}

export interface AIEditAction {
  type: AIEditActionType
  params: Record<string, unknown>
}
```

```
  description: string
  confidence: number
  clipIds?: string[]
}

export type AIEditActionType =
  | 'cut-silence'
  | 'split-clip'
  | 'delete-clip'
  | 'trim-clip'
  | 'add-transition'
  | 'adjust-speed'
  | 'reorder-clips'
  | 'add-subtitle'
  | 'add-marker'
  | 'set-volume'
  | 'detect-scenes'
  | 'generate-highlights'

export interface Transcript {
  mediaId: string
  language: string
  segments: TranscriptSegment[]
  fullText: string
  generatedAt: number
}

export interface TranscriptSegment {
  id: string
  start: number
  end: number
  text: string
  confidence: number
  speaker?: string
  words?: TranscriptWord[]
}

export interface TranscriptWord {
  word: string
  start: number
  end: number
  confidence: number
}

export interface SilenceRegion {
  start: number
  end: number
  duration: number
  averageDb: number
}

export interface SceneDetectionResult {
  mediaId: string
  scenes: SceneBoundary[]
}

export interface SceneBoundary {
  time: number
  score: number
}
```

```
    type: 'cut' | 'gradual'
    thumbnailPath: string | null
  }
}
```

Create `packages/shared-types/src/export.ts`:

typescript

```
export interface ExportJob {
  id: string
  projectId: string
  timelineId: string
  outputPath: string
  preset: ExportPreset
  status: ExportStatus
  progress: number // 0-100
  currentFrame: number
  totalFrames: number
  startedAt: number | null
  completedAt: number | null
  error: string | null
  estimatedTimeRemaining: number | null
}

export type ExportStatus = 'queued' | 'preparing' | 'rendering' |
'encoding' | 'completed' | 'failed' | 'cancelled'

export interface ExportPreset {
  id: string
  name: string
  container: ExportContainer
  videoCodec: VideoCodec
  audioCodec: AudioCodec
  width: number
  height: number
  frameRate: number
  videoBitrate: number // kbps
  audioBitrate: number // kbps
  quality: number // 0-100 (CRF-based)
  hardwareAcceleration: boolean
  customFFmpegArgs: string[]
}

export type ExportContainer = 'mp4' | 'mov' | 'mkv' | 'webm' | 'avi' |
'prores'
export type VideoCodec = 'h264' | 'h265' | 'vp9' | 'av1' | 'prores422' |
'prores4444'
export type AudioCodec = 'aac' | 'mp3' | 'opus' | 'pcm' | 'flac'

export const DEFAULT_EXPORT_PRESETS: ExportPreset[] = [
  {
    id: 'h264-1080p',
    name: 'H.264 1080p (YouTube)',
    container: 'mp4',
    videoCodec: 'h264',
    audioCodec: 'aac',
    width: 1920,
    height: 1080,
    frameRate: 30,
    videoBitrate: 8000,
    audioBitrate: 192,
```

```

    quality: 23,
    hardwareAcceleration: false,
    customFFmpegArgs: [],
  },
  {
    id: 'h265-4k',
    name: 'H.265 4K (Master)',
    container: 'mp4',
    videoCodec: 'h265',
    audioCodec: 'aac',
    width: 3840,
    height: 2160,
    frameRate: 30,
    videoBitrate: 20000,
    audioBitrate: 256,
    quality: 18,
    hardwareAcceleration: false,
    customFFmpegArgs: [],
  },
  {
    id: 'prores-proxy',
    name: 'ProRes 422 Proxy',
    container: 'mov',
    videoCodec: 'prores422',
    audioCodec: 'pcm',
    width: 1920,
    height: 1080,
    frameRate: 30,
    videoBitrate: 45000,
    audioBitrate: 1536,
    quality: 0,
    hardwareAcceleration: false,
    customFFmpegArgs: ['-profile:v', '0'],
  },
]

```

PHASE 2 — FRONTEND APPLICATION ENTRY

Create `apps/desktop/index.html`:

html

```

<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <meta http-equiv="Content-Security-Policy" content="default-src 'self' 'unsafe-inline' 'unsafe-eval' blob: data: http://localhost:7001 http://localhost:7002 http://localhost:11434; media-src 'self' blob: file:; img-src 'self' blob: data: file:;" />
    <title>AXEW</title>
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link href="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700&family=JetBrains+Mono:wght@400;500&display=swap" rel="stylesheet">

```

```
</head>
<body>
  <div id="root"></div>
  <script type="module" src="/src/main.tsx"></script>
</body>
</html>
```

Create `apps/desktop/src/main.tsx`:

tsx

```
import React from 'react'
import ReactDOM from 'react-dom/client'
import { QueryClient, QueryClientProvider } from '@tanstack/react-query'
import App from './App'
import './styles/globals.css'

const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
      retry: 1,
      staleTime: 30000,
    },
  },
})

ReactDOM.createRoot(document.getElementById('root')!).render(
  <React.StrictMode>
    <QueryClientProvider client={queryClient}>
      <App />
    </QueryClientProvider>
  </React.StrictMode>
)
```

Create `apps/desktop/src/styles/globals.css`:

CSS

```
@tailwind base;
@tailwind components;
@tailwind utilities;

@layer base {
  * {
    box-sizing: border-box;
    margin: 0;
    padding: 0;
  }

  html, body, #root {
    width: 100%;
    height: 100%;
    overflow: hidden;
    background-color: #0A0A0C;
    color: #E8E8F0;
    font-family: 'Inter', system-ui, sans-serif;
    font-size: 13px;
    line-height: 1.5;
    -webkit-font-smoothing: antialiased;
    -moz-osx-font-smoothing: grayscale;
    user-select: none;
  }
}
```

```
::-webkit-scrollbar {  
  width: 6px;  
  height: 6px;  
}  
  
::-webkit-scrollbar-track {  
  background: transparent;  
}  
  
::-webkit-scrollbar-thumb {  
  background: #22222A;  
  border-radius: 3px;  
}  
  
::-webkit-scrollbar-thumb:hover {  
  background: #2E2E3A;  
}  
  
::-webkit-scrollbar-corner {  
  background: transparent;  
}  
}  
  
@layer utilities {  
  .text-gradient-accent {  
    background: linear-gradient(135deg, #5B5BFF, #A855F7);  
    -webkit-background-clip: text;  
    -webkit-text-fill-color: transparent;  
    background-clip: text;  
  }  
  
  .panel-bg {  
    background-color: #16161B;  
    border: 1px solid #22222A;  
  }  
  
  .panel-surface {  
    background-color: #111115;  
  }  
  
  .resize-handle-x {  
    cursor: col-resize;  
    width: 4px;  
    background: transparent;  
    transition: background 0.15s;  
  }  
  
  .resize-handle-x:hover {  
    background: #5B5BFF44;  
  }  
  
  .resize-handle-y {  
    cursor: row-resize;  
    height: 4px;  
    background: transparent;  
    transition: background 0.15s;  
  }  
  
  .resize-handle-y:hover {
```

```

    background: #5B5BFF44;
  }

  .no-drag {
    -webkit-app-region: no-drag;
  }

  .drag-region {
    -webkit-app-region: drag;
  }

  .clip-shadow {
    box-shadow: 0 2px 8px rgba(0,0,0,0.5), inset 0 1px 0
    rgba(255,255,255,0.05);
  }
}

```

PHASE 2 — ZUSTAND STATE MANAGEMENT

Create `apps/desktop/src/stores/index.ts`:

typescript

```

export { useProjectStore } from './projectStore'
export { useTimelineStore } from './timelineStore'
export { usePlaybackStore } from './playbackStore'
export { useUIStore } from './uiStore'
export { useAISTore } from './aiStore'
export { useExportStore } from './exportStore'

```

Create `apps/desktop/src/stores/projectStore.ts`:

typescript

```

import { create } from 'zustand'
import { immer } from 'zustand/middleware/immer'
import { subscribeWithSelector } from 'zustand/middleware'
import type { Project, RecentProject, ProjectSettings } from
'@shared/project'
import type { MediaFile } from '@shared/media'
import { createDefaultProject } from '../lib/projectFactory'

interface ProjectState {
  currentProject: Project | null
  recentProjects: RecentProject[]
  isLoading: boolean
  isDirty: boolean
  error: string | null
}

interface ProjectActions {
  createProject: (name: string, settings?: Partial<ProjectSettings>) =>
void
  loadProject: (projectPath: string) => Promise<void>
  saveProject: () => Promise<void>
  closeProject: () => void
  addMediaFile: (file: MediaFile) => void
  removeMediaFile: (fileId: string) => void
  updateMediaFile: (fileId: string, updates: Partial<MediaFile>) => void
  setDirty: (dirty: boolean) => void
}

```

```

clearError: () => void
}

export const useProjectStore = create<ProjectState & ProjectActions>() (
  subscribeWithSelector(
    immer((set, get) => ({
      currentProject: null,
      recentProjects: [],
      isLoading: false,
      isDirty: false,
      error: null,

      createProject: (name, settings) => {
        const project = createDefaultProject(name, settings)
        set((state) => {
          state.currentProject = project
          state.isDirty = false
          state.error = null
        })
      },

      loadProject: async (projectPath) => {
        set((state) => { state.isLoading = true; state.error = null })
        try {
          const result = await window.axew.fs.readFile(projectPath)
          if (!result.success) throw new Error(result.error)
          const data = atob(result.data)
          const project: Project = JSON.parse(data)
          set((state) => {
            state.currentProject = project
            state.isLoading = false
            state.isDirty = false
          })
        } catch (err) {
          set((state) => {
            state.error = String(err)
            state.isLoading = false
          })
        }
      },

      saveProject: async () => {
        const { currentProject } = get()
        if (!currentProject) return
        try {
          const data = JSON.stringify(currentProject, null, 2)
          const result = await
window.axew.fs.writeFile(currentProject.path, data)
          if (!result.success) throw new Error(result.error)
          set((state) => { state.isDirty = false })
        } catch (err) {
          set((state) => { state.error = String(err) })
        }
      },

      closeProject: () => {
        set((state) => {
          state.currentProject = null
          state.isDirty = false

```

```

        state.error = null
    })
  },

  addMediaFile: (file) => {
    set((state) => {
      if (state.currentProject) {
        state.currentProject.mediaFiles[file.id] = file
        state.currentProject.mediaBin.rootFolder.mediaIds.push(file.id)
        state.isDirty = true
      }
    })
  },

  removeMediaFile: (fileId) => {
    set((state) => {
      if (state.currentProject) {
        delete state.currentProject.mediaFiles[fileId]
        const folder = state.currentProject.mediaBin.rootFolder
        folder.mediaIds = folder.mediaIds.filter(id => id !== fileId)
        state.isDirty = true
      }
    })
  },

  updateMediaFile: (fileId, updates) => {
    set((state) => {
      if (state.currentProject?.mediaFiles[fileId]) {
        Object.assign(state.currentProject.mediaFiles[fileId], updates)
        state.isDirty = true
      }
    })
  },

  setDirty: (dirty) => {
    set((state) => { state.isDirty = dirty })
  },

  clearError: () => {
    set((state) => { state.error = null })
  },
}))
)
]

```

Create `apps/desktop/src/stores/timelineStore.ts`:

typescript

```

import { create } from 'zustand'
import { immer } from 'zustand/middleware/immer'
import { subscribeWithSelector } from 'zustand/middleware'
import { nanoid } from 'nanoid'
import type { Timeline, Track, Clip, Marker, TrackType } from
'@shared/timeline'
import { useProjectStore } from '../projectStore'

interface TimelineState {
  selectedClipIds: string[]
  selectedTrackId: string | null
  hoveredClipId: string | null

```

```

    zoom: number // pixels per second
    scrollX: number // horizontal scroll offset
    scrollY: number // vertical scroll offset
    snapEnabled: boolean
    rippleEnabled: boolean
    undoStack: TimelineSnapshot[]
    redoStack: TimelineSnapshot[]
  }

  interface TimelineSnapshot {
    timestamp: number
    description: string
    timeline: Timeline
  }

  interface TimelineActions {
    selectClip: (clipId: string, multi?: boolean) => void
    deselectAll: () => void
    setZoom: (zoom: number) => void
    setScrollX: (x: number) => void
    setScrollY: (y: number) => void
    toggleSnap: () => void
    toggleRipple: () => void

    // Track operations
    addTrack: (type: TrackType, name?: string) => void
    deleteTrack: (trackId: string) => void
    updateTrack: (trackId: string, updates: Partial<Track>) => void
    moveTrack: (trackId: string, newIndex: number) => void

    // Clip operations
    addClip: (trackId: string, clip: Omit<Clip, 'id' | 'trackId'>) => string
    deleteClip: (clipId: string) => void
    updateClip: (clipId: string, updates: Partial<Clip>) => void
    moveClip: (clipId: string, trackId: string, startTime: number) => void
    splitClip: (clipId: string, time: number) => void
    trimClipIn: (clipId: string, newInPoint: number) => void
    trimClipOut: (clipId: string, newOutPoint: number) => void

    // Marker operations
    addMarker: (marker: Omit<Marker, 'id'>) => void
    deleteMarker: (markerId: string) => void
    updateMarker: (markerId: string, updates: Partial<Marker>) => void

    // History
    pushUndoSnapshot: (description: string) => void
    undo: () => void
    redo: () => void
  }

  const getTimeline = (): Timeline | null => {
    return useProjectStore.getState().currentProject?.timeline ?? null
  }

  const setTimeline = (updater: (timeline: Timeline) => void) => {
    useProjectStore.setState((state) => {
      if (state.currentProject) {
        updater(state.currentProject.timeline)
        state.isDirty = true
      }
    })
  }

```

```

    }
  })
}

export const useTimelineStore = create<TimelineState & TimelineActions>() (
  subscribeWithSelector(
    immer((set, get) => ({
      selectedClipIds: [],
      selectedTrackId: null,
      hoveredClipId: null,
      zoom: 100,
      scrollX: 0,
      scrollY: 0,
      snapEnabled: true,
      rippleEnabled: false,
      undoStack: [],
      redoStack: [],

      selectClip: (clipId, multi = false) => {
        set((state) => {
          if (multi) {
            const idx = state.selectedClipIds.indexOf(clipId)
            if (idx >= 0) {
              state.selectedClipIds.splice(idx, 1)
            } else {
              state.selectedClipIds.push(clipId)
            }
          } else {
            state.selectedClipIds = [clipId]
          }
        })
      },

      deselectAll: () => {
        set((state) => {
          state.selectedClipIds = []
          state.selectedTrackId = null
        })
      },

      setZoom: (zoom) => set((state) => { state.zoom = Math.max(10,
Math.min(1000, zoom)) }),
      setScrollX: (x) => set((state) => { state.scrollX = Math.max(0, x)
}),
      setScrollY: (y) => set((state) => { state.scrollY = Math.max(0, y)
}),
      toggleSnap: () => set((state) => { state.snapEnabled =
!state.snapEnabled }),
      toggleRipple: () => set((state) => { state.rippleEnabled =
!state.rippleEnabled }),

      addTrack: (type, name) => {
        const id = nanoid()
        setTimeline((timeline) => {
          const newTrack: Track = {
            id,
            timelineId: timeline.id,
            name: name || `${type.charAt(0).toUpperCase() + type.slice(1)}
${timeline.tracks.filter(t => t.type === type).length + 1}`,

```

```

      type,
      index: timeline.tracks.length,
      height: type === 'video' ? 80 : 60,
      locked: false,
      muted: false,
      visible: true,
      solo: false,
      color: type === 'video' ? '#1E3A5F' : type === 'audio' ?
'#1E4F3A' : '#4F1E3A',
      clips: [],
    }
    timeline.tracks.push(newTrack)
  })
},

  deleteTrack: (trackId) => {
    setTimeline((timeline) => {
      timeline.tracks = timeline.tracks.filter(t => t.id !== trackId)
      timeline.tracks.forEach((t, i) => { t.index = i })
    })
  },

  updateTrack: (trackId, updates) => {
    setTimeline((timeline) => {
      const track = timeline.tracks.find(t => t.id === trackId)
      if (track) Object.assign(track, updates)
    })
  },

  moveTrack: (trackId, newIndex) => {
    setTimeline((timeline) => {
      const idx = timeline.tracks.findIndex(t => t.id === trackId)
      if (idx < 0) return
      const [track] = timeline.tracks.splice(idx, 1)
      timeline.tracks.splice(newIndex, 0, track)
      timeline.tracks.forEach((t, i) => { t.index = i })
    })
  },

  addClip: (trackId, clipData) => {
    const id = nanoid()
    setTimeline((timeline) => {
      const track = timeline.tracks.find(t => t.id === trackId)
      if (track) {
        const clip: Clip = {
          id,
          trackId,
          ...clipData,
          effects: clipData.effects ?? [],
          transitions: clipData.transitions ?? { in: null, out: null },
          keyframes: clipData.keyframes ?? [],
        }
        track.clips.push(clip)
        track.clips.sort((a, b) => a.startTime - b.startTime)
      }
    })
    return id
  },
},

```

```

    deleteClip: (clipId) => {
      setTimeline((timeline) => {
        for (const track of timeline.tracks) {
          const idx = track.clips.findIndex(c => c.id === clipId)
          if (idx >= 0) {
            track.clips.splice(idx, 1)
            break
          }
        }
      })
      set((state) => {
        state.selectedClipIds = state.selectedClipIds.filter(id => id !== clipId)
      })
    },

    updateClip: (clipId, updates) => {
      setTimeline((timeline) => {
        for (const track of timeline.tracks) {
          const clip = track.clips.find(c => c.id === clipId)
          if (clip) {
            Object.assign(clip, updates)
            track.clips.sort((a, b) => a.startTime - b.startTime)
            break
          }
        }
      })
    },

    moveClip: (clipId, targetTrackId, startTime) => {
      setTimeline((timeline) => {
        let foundClip: Clip | null = null
        for (const track of timeline.tracks) {
          const idx = track.clips.findIndex(c => c.id === clipId)
          if (idx >= 0) {
            [foundClip] = track.clips.splice(idx, 1)
            break
          }
        }
        if (foundClip) {
          const targetTrack = timeline.tracks.find(t => t.id === targetTrackId)
          if (targetTrack) {
            foundClip.trackId = targetTrackId
            foundClip.startTime = Math.max(0, startTime)
            targetTrack.clips.push(foundClip)
            targetTrack.clips.sort((a, b) => a.startTime - b.startTime)
          }
        }
      })
    },

    splitClip: (clipId, time) => {
      const newId = nanoid()
      setTimeline((timeline) => {
        for (const track of timeline.tracks) {
          const clip = track.clips.find(c => c.id === clipId)
          if (!clip) continue

```

```

        if (time <= clip.startTime || time >= clip.startTime +
clip.duration) return
        const splitPoint = time - clip.startTime
        const originalDuration = clip.duration
        const originalOutPoint = clip.mediaOutPoint
        clip.duration = splitPoint
        clip.mediaOutPoint = clip.mediaInPoint + splitPoint
        const newClip: Clip = {
            ...JSON.parse(JSON.stringify(clip)),
            id: newId,
            startTime: time,
            duration: originalDuration - splitPoint,
            mediaInPoint: clip.mediaInPoint + splitPoint,
            mediaOutPoint: originalOutPoint,
        }
        track.clips.push(newClip)
        track.clips.sort((a, b) => a.startTime - b.startTime)
        break
    }
    })
},

trimClipIn: (clipId, deltaTime) => {
    setTimeline((timeline) => {
        for (const track of timeline.tracks) {
            const clip = track.clips.find(c => c.id === clipId)
            if (!clip) continue
            const newStartTime = clip.startTime + deltaTime
            const newMediaIn = clip.mediaInPoint + deltaTime
            const minDuration = 1 / 30
            if (clip.duration - deltaTime < minDuration) return
            if (newMediaIn < 0) return
            clip.startTime = newStartTime
            clip.mediaInPoint = newMediaIn
            clip.duration -= deltaTime
            break
        }
    })
},

trimClipOut: (clipId, deltaTime) => {
    setTimeline((timeline) => {
        for (const track of timeline.tracks) {
            const clip = track.clips.find(c => c.id === clipId)
            if (!clip) continue
            const newDuration = clip.duration + deltaTime
            const newMediaOut = clip.mediaOutPoint + deltaTime
            const minDuration = 1 / 30
            if (newDuration < minDuration) return
            clip.duration = newDuration
            clip.mediaOutPoint = newMediaOut
            break
        }
    })
},

addMarker: (markerData) => {
    const id = nanoid()
    setTimeline((timeline) => {

```

```

        timeline.markers.push({ id, ...markerData })
        timeline.markers.sort((a, b) => a.time - b.time)
    })
},

    deleteMarker: (markerId) => {
        setTimeline((timeline) => {
            timeline.markers = timeline.markers.filter(m => m.id !==
markerId)
        })
    },

    updateMarker: (markerId, updates) => {
        setTimeline((timeline) => {
            const marker = timeline.markers.find(m => m.id === markerId)
            if (marker) Object.assign(marker, updates)
        })
    },

    pushUndoSnapshot: (description) => {
        const timeline = getTimeline()
        if (!timeline) return
        set((state) => {
            state.undoStack.push({
                timestamp: Date.now(),
                description,
                timeline: JSON.parse(JSON.stringify(timeline)),
            })
            if (state.undoStack.length > 50) state.undoStack.shift()
            state.redoStack = []
        })
    },

    undo: () => {
        const { undoStack } = get()
        if (undoStack.length === 0) return
        const currentTimeline = getTimeline()
        if (!currentTimeline) return
        set((state) => {
            const snapshot = state.undoStack.pop()!
            state.redoStack.push({
                timestamp: Date.now(),
                description: 'Redo',
                timeline: JSON.parse(JSON.stringify(currentTimeline)),
            })
            setTimeline((tl) => Object.assign(tl, snapshot.timeline))
        })
    },

    redo: () => {
        const { redoStack } = get()
        if (redoStack.length === 0) return
        const currentTimeline = getTimeline()
        if (!currentTimeline) return
        set((state) => {
            const snapshot = state.redoStack.pop()!
            state.undoStack.push({
                timestamp: Date.now(),
                description: 'Undo',

```

```

        timeline: JSON.parse(JSON.stringify(currentTimeline)),
    })
    setTimeline((tl) => Object.assign(tl, snapshot.timeline))
  })
},
)))
)
]

```

Create `apps/desktop/src/stores/playbackStore.ts`:

typescript

```

import { create } from 'zustand'
import { subscribeWithSelector } from 'zustand/middleware'
import type { PlayheadState } from '@shared/timeline'

interface PlaybackState extends PlayheadState {
  videoRef: HTMLVideoElement | null
  previewReady: boolean
  frameRate: number
  duration: number
  volume: number
  muted: boolean
  buffering: boolean
}

interface PlaybackActions {
  setCurrentTime: (time: number) => void
  play: () => void
  pause: () => void
  togglePlay: () => void
  stop: () => void
  seekToFrame: (frame: number) => void
  nextFrame: () => void
  previousFrame: () => void
  setVolume: (volume: number) => void
  setMuted: (muted: boolean) => void
  setLoop: (loop: boolean) => void
  setLoopPoints: (inPoint: number | null, outPoint: number | null) => void
  setSpeed: (speed: number) => void
  setDuration: (duration: number) => void
  setFrameRate: (fps: number) => void
  setVideoRef: (ref: HTMLVideoElement | null) => void
  setPreviewReady: (ready: boolean) => void
  setBuffering: (buffering: boolean) => void
}

export const usePlaybackStore = create<PlaybackState & PlaybackActions>() (
  subscribeWithSelector((set, get) => ({
    currentTime: 0,
    playing: false,
    loop: false,
    loopIn: null,
    loopOut: null,
    speed: 1,
    videoRef: null,
    previewReady: false,
    frameRate: 30,
    duration: 0,
    volume: 1,

```

```

    muted: false,
    buffering: false,

    setCurrentTime: (time) => {
      const { duration, videoRef } = get()
      const clampedTime = Math.max(0, Math.min(duration, time))
      set({ currentTime: clampedTime })
      if (videoRef) videoRef.currentTime = clampedTime
    },

    play: () => {
      const { videoRef, speed } = get()
      set({ playing: true })
      if (videoRef) {
        videoRef.playbackRate = speed
        videoRef.play().catch(() => set({ playing: false }))
      }
    },

    pause: () => {
      const { videoRef } = get()
      set({ playing: false })
      videoRef?.pause()
    },

    togglePlay: () => {
      const { playing } = get()
      if (playing) get().pause()
      else get().play()
    },

    stop: () => {
      const { videoRef } = get()
      set({ playing: false, currentTime: 0 })
      if (videoRef) {
        videoRef.pause()
        videoRef.currentTime = 0
      }
    },

    seekToFrame: (frame) => {
      const { frameRate } = get()
      get().setCurrentTime(frame / frameRate)
    },

    nextFrame: () => {
      const { currentTime, frameRate } = get()
      get().pause()
      get().setCurrentTime(currentTime + 1 / frameRate)
    },

    previousFrame: () => {
      const { currentTime, frameRate } = get()
      get().pause()
      get().setCurrentTime(currentTime - 1 / frameRate)
    },

    setVolume: (volume) => {
      const { videoRef } = get()

```

```

        set({ volume: Math.max(0, Math.min(1, volume)) })
        if (videoRef) videoRef.volume = volume
    },

    setMuted: (muted) => {
        const { videoRef } = get()
        set({ muted })
        if (videoRef) videoRef.muted = muted
    },

    setLoop: (loop) => set({ loop }),

    setLoopPoints: (inPoint, outPoint) => set({ loopIn: inPoint, loopOut:
outPoint }),

    setSpeed: (speed) => {
        const { videoRef } = get()
        set({ speed })
        if (videoRef) videoRef.playbackRate = speed
    },

    setDuration: (duration) => set({ duration }),
    setFrameRate: (fps) => set({ frameRate: fps }),
    setVideoRef: (ref) => set({ videoRef: ref }),
    setPreviewReady: (ready) => set({ previewReady: ready }),
    setBuffering: (buffering) => set({ buffering }),
    )))
}

```

Create `apps/desktop/src/stores/uiStore.ts`:

typescript

```

import { create } from 'zustand'
import { persist } from 'zustand/middleware'

export type PanelId = 'mediaBin' | 'inspector' | 'ai' | 'export' |
'effects'

interface PanelLayout {
    leftWidth: number
    rightWidth: number
    timelineHeight: number
    aiPanelOpen: boolean
    inspectorPanelOpen: boolean
}

interface UIState {
    layout: PanelLayout
    activePanel: PanelId | null
    showWelcome: boolean
    showSettings: boolean
    showExportDialog: boolean
    showShortcutsOverlay: boolean
    theme: 'dark'
    notifications: Notification[]
    statusMessage: string | null
    isMediaImporting: boolean
}

interface Notification {

```

```

    id: string
    type: 'info' | 'success' | 'warning' | 'error'
    message: string
    timestamp: number
    duration?: number
  }
}

interface UIActions {
  setLayout: (layout: Partial<PanelLayout>) => void
  togglePanel: (panelId: PanelId) => void
  setActivePanel: (panelId: PanelId | null) => void
  setShowWelcome: (show: boolean) => void
  setShowSettings: (show: boolean) => void
  setShowExportDialog: (show: boolean) => void
  setShowShortcutsOverlay: (show: boolean) => void
  addNotification: (notification: Omit<Notification, 'id' | 'timestamp'>)
=> void
  removeNotification: (id: string) => void
  setStatusMessage: (message: string | null) => void
  setIsMediaImporting: (importing: boolean) => void
}

const generateId = () => Math.random().toString(36).slice(2)

export const useUIStore = create<UIState & UIActions>() (
  persist(
    (set, get) => ({
      layout: {
        leftWidth: 260,
        rightWidth: 280,
        timelineHeight: 320,
        aiPanelOpen: false,
        inspectorPanelOpen: true,
      },
      activePanel: 'mediaBin',
      showWelcome: true,
      showSettings: false,
      showExportDialog: false,
      showShortcutsOverlay: false,
      theme: 'dark',
      notifications: [],
      statusMessage: null,
      isMediaImporting: false,

      setLayout: (layout) => set((state) => ({
        layout: { ...state.layout, ...layout }
      })),

      togglePanel: (panelId) => set((state) => {
        if (panelId === 'ai') {
          return { layout: { ...state.layout, aiPanelOpen:
!state.layout.aiPanelOpen } }
        }
        if (panelId === 'inspector') {
          return { layout: { ...state.layout, inspectorPanelOpen:
!state.layout.inspectorPanelOpen } }
        }
        return { activePanel: state.activePanel === panelId ? null :
panelId }
      })
    )
  )
)

```

```

    })),
    setActivePanel: (panelId) => set({ activePanel: panelId }),
    setShowWelcome: (show) => set({ showWelcome: show }),
    setShowSettings: (show) => set({ showSettings: show }),
    setShowExportDialog: (show) => set({ showExportDialog: show }),
    setShowShortcutsOverlay: (show) => set({ showShortcutsOverlay: show
  })),

  addNotification: (notification) => {
    const id = generateId()
    set((state) => ({
      notifications: [...state.notifications, { id, timestamp:
Date.now(), ...notification }])
    )))
    const duration = notification.duration ?? 4000
    if (duration > 0) {
      setTimeout(() => get().removeNotification(id), duration)
    }
    return id
  },

  removeNotification: (id) => set((state) => ({
    notifications: state.notifications.filter(n => n.id !== id)
  })),

  setStatusMessage: (message) => set({ statusMessage: message }),
  setIsMediaImporting: (importing) => set({ isMediaImporting: importing
})),
  })),
  {
    name: 'axew-ui-layout',
    partialize: (state) => ({ layout: state.layout }),
  }
)
]

```

Create `apps/desktop/src/stores/aiStore.ts`:

typescript

```

import { create } from 'zustand'
import { subscribeWithSelector } from 'zustand/middleware'
import type { AIMessage, AIModel, AIEditAction } from '@shared/ai'

interface AIState {
  models: AIModel[]
  activeModel: string | null
  messages: AIMessage[]
  isThinking: boolean
  pendingActions: AIEditAction[]
  streamingContent: string
  ollamaStatus: 'connected' | 'disconnected' | 'checking'
  aiServiceStatus: 'connected' | 'disconnected' | 'checking'
}

interface AIActions {
  setModels: (models: AIModel[]) => void
  setActiveModel: (modelId: string) => void
  addMessage: (message: Omit<AIMessage, 'id' | 'timestamp'>) => void
  clearMessages: () => void
}

```

```

    setIsThinking: (thinking: boolean) => void
    appendStreamingContent: (chunk: string) => void
    finalizeStreamingMessage: () => void
    addPendingAction: (action: AIEditAction) => void
    clearPendingActions: () => void
    removePendingAction: (index: number) => void
    setOllamaStatus: (status: AIState['ollamaStatus']) => void
    setAIServiceStatus: (status: AIState['aiServiceStatus']) => void
  }

const genId = () => Math.random().toString(36).slice(2)

export const useAIStore = create<AIState & AIActions>()({
  subscribeWithSelector((set, get) => ({
    models: [],
    activeModel: null,
    messages: [],
    isThinking: false,
    pendingActions: [],
    streamingContent: '',
    ollamaStatus: 'checking',
    aiServiceStatus: 'checking',

    setModels: (models) => set({ models }),
    setActiveModel: (modelId) => set({ activeModel: modelId }),

    addMessage: (message) => {
      set((state) => ({
        messages: [...state.messages, { id: genId(), timestamp: Date.now(),
...message }],
      })))
    },

    clearMessages: () => set({ messages: [] }),
    setIsThinking: (thinking) => set({ isThinking: thinking }),

    appendStreamingContent: (chunk) => {
      set((state) => ({ streamingContent: state.streamingContent + chunk
    })))
    },

    finalizeStreamingMessage: () => {
      const { streamingContent } = get()
      if (streamingContent) {
        set((state) => ({
          messages: [...state.messages, {
            id: genId(),
            timestamp: Date.now(),
            role: 'assistant' as const,
            content: streamingContent,
          }],
          streamingContent: '',
          isThinking: false,
        })))
      }
    },

    addPendingAction: (action) => {

```

```

        set((state) => ({ pendingActions: [...state.pendingActions, action]
    })))
    },

    clearPendingActions: () => set({ pendingActions: [] }),

    removePendingAction: (index) => {
        set((state) => ({
            pendingActions: state.pendingActions.filter((_, i) => i !== index)
        })))
    },

    setOllamaStatus: (status) => set({ ollamaStatus: status }),
    setAIServiceStatus: (status) => set({ aiServiceStatus: status }),
    })))
}

```

Create `apps/desktop/src/stores/exportStore.ts`:

typescript

```

import { create } from 'zustand'
import type { ExportJob, ExportPreset } from '@shared/export'
import { DEFAULT_EXPORT_PRESETS } from '@shared/export'

interface ExportState {
    jobs: ExportJob[]
    activeJobId: string | null
    presets: ExportPreset[]
    selectedPresetId: string
}

interface ExportActions {
    addJob: (job: ExportJob) => void
    updateJob: (jobId: string, updates: Partial<ExportJob>) => void
    removeJob: (jobId: string) => void
    setActiveJob: (jobId: string | null) => void
    setSelectedPreset: (presetId: string) => void
    addPreset: (preset: ExportPreset) => void
}

export const useExportStore = create<ExportState & ExportActions>()((set)
=> ({
    jobs: [],
    activeJobId: null,
    presets: DEFAULT_EXPORT_PRESETS,
    selectedPresetId: 'h264-1080p',

    addJob: (job) => set((state) => ({ jobs: [...state.jobs, job] })),
    updateJob: (jobId, updates) => set((state) => ({
        jobs: state.jobs.map(j => j.id === jobId ? { ...j, ...updates } : j)
    })),
    removeJob: (jobId) => set((state) => ({ jobs: state.jobs.filter(j => j.id
    !== jobId) })),
    setActiveJob: (jobId) => set({ activeJobId: jobId }),
    setSelectedPreset: (presetId) => set({ selectedPresetId: presetId }),
    addPreset: (preset) => set((state) => ({ presets: [...state.presets,
    preset] })),
    })))

```

PHASE 2 — MAIN APP COMPONENT & LAYOUT

Create `apps/desktop/src/App.tsx`:

`tsx`

```
import React, { useEffect } from 'react'
import { AnimatePresence, motion } from 'framer-motion'
import { useUIStore } from '../stores/uiStore'
import { useProjectStore } from '../stores/projectStore'
import { MainLayout } from '../components/layout/MainLayout'
import { WelcomeScreen } from '../components/welcome/WelcomeScreen'
import { NotificationStack } from '../components/ui/NotificationStack'
import { useKeyboardShortcuts } from '../hooks/useKeyboardShortcuts'
import { useServiceStatus } from '../hooks/useServiceStatus'
import { useMenuEvents } from '../hooks/useMenuEvents'

export default function App() {
  const { showWelcome, setShowWelcome } = useUIStore()
  const { currentProject } = useProjectStore()

  useKeyboardShortcuts()
  useServiceStatus()
  useMenuEvents()

  const showEditor = !showWelcome && currentProject !== null

  return (
    <div className="w-full h-full flex flex-col bg-axew-bg text-axew-text overflow-hidden">
      <AnimatePresence mode="wait">
        {showWelcome || !currentProject ? (
          <motion.div
            key="welcome"
            initial={{ opacity: 0 }}
            animate={{ opacity: 1 }}
            exit={{ opacity: 0 }}
            transition={{ duration: 0.2 }}
            className="w-full h-full"
          >
            <WelcomeScreen onClose={() => setShowWelcome(false)} />
          </motion.div>
        ) : (
          <motion.div
            key="editor"
            initial={{ opacity: 0 }}
            animate={{ opacity: 1 }}
            exit={{ opacity: 0 }}
            transition={{ duration: 0.15 }}
            className="w-full h-full"
          >
            <MainLayout />
          </motion.div>
        )}
      </AnimatePresence>
      <NotificationStack />
    </div>
  )
}
```

Create apps/desktop/src/components/layout/MainLayout.tsx:

tsx

```
import React, { useRef, useState, useCallback } from 'react'
import { TitleBar } from '../TitleBar'
import { Toolbar } from '../toolbar/Toolbar'
import { MediaBinPanel } from '../panels/MediaBinPanel'
import { PreviewMonitor } from '../preview/PreviewMonitor'
import { InspectorPanel } from '../panels/InspectorPanel'
import { Timeline } from '../timeline/Timeline'
import { AIPanel } from '../panels/AIPanel'
import { StatusBar } from '../StatusBar'
import { useUIStore } from '../../stores/uiStore'

export function MainLayout() {
  const { layout, setLayout } = useUIStore()
  const containerRef = useRef<HTMLDivElement>(null)

  const [isResizingLeft, setIsResizingLeft] = useState(false)
  const [isResizingRight, setIsResizingRight] = useState(false)
  const [isResizingTimeline, setIsResizingTimeline] = useState(false)

  const handleLeftResize = useCallback((e: React.MouseEvent) => {
    e.preventDefault()
    setIsResizingLeft(true)
    const startX = e.clientX
    const startWidth = layout.leftWidth

    const onMove = (me: MouseEvent) => {
      const delta = me.clientX - startX
      setLayout({ leftWidth: Math.max(180, Math.min(500, startWidth +
delta)) })
    }

    const onUp = () => {
      setIsResizingLeft(false)
      document.removeEventListener('mousemove', onMove)
      document.removeEventListener('mouseup', onUp)
    }

    document.addEventListener('mousemove', onMove)
    document.addEventListener('mouseup', onUp)
  }, [layout.leftWidth, setLayout])

  const handleRightResize = useCallback((e: React.MouseEvent) => {
    e.preventDefault()
    setIsResizingRight(true)
    const startX = e.clientX
    const startWidth = layout.rightWidth

    const onMove = (me: MouseEvent) => {
      const delta = startX - me.clientX
      setLayout({ rightWidth: Math.max(200, Math.min(500, startWidth +
delta)) })
    }

    const onUp = () => {
      setIsResizingRight(false)
      document.removeEventListener('mousemove', onMove)
    }
  })
}
```

```

        document.removeEventListener('mouseup', onUp)
    }

    document.addEventListener('mousemove', onMove)
    document.addEventListener('mouseup', onUp)
    }, [layout.rightWidth, setLayout])

    const handleTimelineResize = useCallback((e: React.MouseEvent) => {
        e.preventDefault()
        setIsResizingTimeline(true)
        const startY = e.clientY
        const startHeight = layout.timelineHeight

        const onMove = (me: MouseEvent) => {
            const delta = startY - me.clientY
            setLayout({ timelineHeight: Math.max(180, Math.min(600, startHeight +
delta)) })
        }

        const onUp = () => {
            setIsResizingTimeline(false)
            document.removeEventListener('mousemove', onMove)
            document.removeEventListener('mouseup', onUp)
        }

        document.addEventListener('mousemove', onMove)
        document.addEventListener('mouseup', onUp)
    }, [layout.timelineHeight, setLayout])

    return (
        <div ref={containerRef} className="flex flex-col w-full h-full
overflow-hidden">
            <TitleBar />
            <Toolbar />

            {/* Main Content Area */}
            <div className="flex flex-1 overflow-hidden" style={{ cursor:
isResizingLeft || isResizingRight ? 'col-resize' : isResizingTimeline ?
'row-resize' : undefined }}>

                {/* Left Panel - Media Bin */}
                <div
                    className="flex flex-col flex-shrink-0 border-r border-axew-
border overflow-hidden"
                    style={{ width: layout.leftWidth }}
                >
                    <MediaBinPanel />
                </div>

                {/* Resize Handle Left */}
                <div
                    className="resize-handle-x flex-shrink-0 z-10"
                    onMouseDown={handleLeftResize}
                />

                {/* Center - Preview + AI */}
                <div className="flex flex-col flex-1 overflow-hidden min-w-0">

                    {/* Preview Row */}

```

```

        <div className="flex flex-1 overflow-hidden min-h-0">
            <div className="flex-1 overflow-hidden min-w-0">
                <PreviewMonitor />
            </div>

            {layout.aiPanelOpen && (
                <>
                    <div className="resize-handle-x flex-shrink-0" />
                    <div className="flex-shrink-0 border-l border-axew-border"
style={{ width: 320 }}>
                        <AIPanel />
                    </div>
                </>
            )}
        </div>

        {/* Timeline Resize Handle */}
        <div
            className="resize-handle-y flex-shrink-0 z-10 border-t border-
axew-border"
            onMouseDown={handleTimelineResize}
        />

        {/* Timeline */}
        <div
            className="flex-shrink-0 overflow-hidden border-t border-axew-
border"
            style={{ height: layout.timelineHeight }}
        >
            <Timeline />
        </div>
    </div>

    {/* Resize Handle Right */}
    {layout.inspectorPanelOpen && (
        <div
            className="resize-handle-x flex-shrink-0 z-10"
            onMouseDown={handleRightResize}
        />
    )}

    {/* Right Panel - Inspector */}
    {layout.inspectorPanelOpen && (
        <div
            className="flex flex-col flex-shrink-0 border-l border-axew-
border overflow-hidden"
            style={{ width: layout.rightWidth }}
        >
            <InspectorPanel />
        </div>
    )}
</div>

<StatusBar />
</div>
)
}

```

Create `apps/desktop/src/components/layout/TitleBar.tsx`:

tsx

```
import React from 'react'
import { useProjectStore } from '../../stores/projectStore'
import { useUIStore } from '../../stores/uiStore'
import { Zap } from 'lucide-react'

export function TitleBar() {
  const { currentProject, isDirty } = useProjectStore()
  const { layout, togglePanel } = useUIStore()

  const isMac = navigator.platform.toLowerCase().includes('mac')

  return (
    <div
      className="drag-region flex items-center h-9 px-3 flex-shrink-0
border-b border-axew-border bg-axew-bg select-none"
      style={{ paddingLeft: isMac ? '80px' : '12px' }}
    >
      { /* App Logo */ }
      <div className="no-drag flex items-center gap-1.5 mr-4">
        <Zap size={14} className="text-axew-accent" />
        <span className="text-xs font-semibold tracking-widest text-axew-
text uppercase">AXEW</span>
      </div>

      { /* Project Title */ }
      <div className="flex-1 flex items-center justify-center">
        {currentProject && (
          <span className="text-xs text-axew-textMuted">
            {currentProject.name}
            {isDirty && <span className="ml-1 text-axew-accent">●</span>}
          </span>
        ) }
      </div>

      { /* Window Controls (non-Mac) */ }
      { !isMac && (
        <div className="no-drag flex items-center gap-1">
          <span className="text-2xs text-axew-textDim font-
mono">v0.1.0</span>
        </div>
      ) }
    </div>
  )
}
```

Create `apps/desktop/src/components/layout/StatusBar.tsx`:

tsx

```
import React, { useEffect, useState } from 'react'
import { usePlaybackStore } from '../../stores/playbackStore'
import { useTimelineStore } from '../../stores/timelineStore'
import { useAIStore } from '../../stores/aiStore'
import { useUIStore } from '../../stores/uiStore'
import { formatTimecode } from '../../lib/timecode'
import { Activity, Cpu, HardDrive, Zap } from 'lucide-react'

export function StatusBar() {
  const { currentTime, duration, frameRate } = usePlaybackStore()
```

```

const { zoom } = useTimelineStore()
const { aiServiceStatus, ollamaStatus } = useAIStore()
const { statusMessage } = useUIStore()

return (
  <div className="flex items-center h-6 px-3 bg-axew-bg border-t border-axew-border flex-shrink-0 text-2xs text-axew-textMuted select-none gap-4">

    {/* Timecode */}
    <div className="flex items-center gap-1.5 font-mono">
      <span>{formatTimecode(currentTime, frameRate)}</span>
      <span className="text-axew-textDim"></span>
      <span className="text-axew-textDim">{formatTimecode(duration, frameRate)}</span>
    </div>

    <div className="h-3 w-px bg-axew-border" />

    {/* Zoom */}
    <span>{zoom}px/s</span>

    <div className="h-3 w-px bg-axew-border" />

    {/* Frame Rate */}
    <span>{frameRate}fps</span>

    {/* Status Message */}
    {statusMessage && (
      <>
        <div className="h-3 w-px bg-axew-border" />
        <span className="text-axew-accent">{statusMessage}</span>
      </>
    )}

    {/* Spacer */}
    <div className="flex-1" />

    {/* Service Status */}
    <div className="flex items-center gap-2">
      <div className="flex items-center gap-1">
        <Zap size={9} className={ollamaStatus === 'connected' ? 'text-axew-success' : 'text-axew-textDim'} />
        <span>Ollama</span>
      </div>
      <div className="flex items-center gap-1">
        <Activity size={9} className={aiServiceStatus === 'connected' ? 'text-axew-success' : 'text-axew-textDim'} />
        <span>AI</span>
      </div>
      <div className="flex items-center gap-1">
        <Cpu size={9} className="text-axew-textDim" />
        <span>GPU</span>
      </div>
    </div>
  </div>
)
}

```

PHASE 2 — TOOLBAR

Create `apps/desktop/src/components/toolbar/Toolbar.tsx`:

tsx

```
import React from 'react'
import { motion } from 'framer-motion'
import {
  MousePointer2, Scissors, Minus, Type, Hand,
  SkipBack, SkipForward, Play, Pause, Square,
  ZoomIn, ZoomOut, Magnet, Layers,
  Undo2, Redo2, Upload, Download, Sparkles,
  ChevronLeft, ChevronRight
} from 'lucide-react'
import { usePlaybackStore } from '../../stores/playbackStore'
import { useTimelineStore } from '../../stores/timelineStore'
import { useUIStore } from '../../stores/uiStore'
import { clsx } from 'clsx'

type Tool = 'select' | 'blade' | 'slip' | 'text' | 'hand'

const tools: { id: Tool; icon: React.ElementType; label: string; shortcut: string }[] = [
  { id: 'select', icon: MousePointer2, label: 'Select', shortcut: 'V' },
  { id: 'blade', icon: Scissors, label: 'Blade', shortcut: 'B' },
  { id: 'slip', icon: Minus, label: 'Slip', shortcut: 'S' },
  { id: 'text', icon: Type, label: 'Text', shortcut: 'T' },
  { id: 'hand', icon: Hand, label: 'Hand', shortcut: 'H' },
]

export function Toolbar() {
  const [activeTool, setActiveTool] = React.useState<Tool>('select')
  const { playing, togglePlay, stop, nextFrame, previousFrame } =
    usePlaybackStore()
  const { zoom, setZoom, snapEnabled, toggleSnap, undo, redo, undoStack,
    redoStack } = useTimelineStore()
  const { setShowExportDialog, togglePanel, layout } = useUIStore()

  const canUndo = undoStack.length > 0
  const canRedo = redoStack.length > 0

  return (
    <div className="flex items-center h-10 px-3 gap-1 flex-shrink-0 border-
b border-axew-border bg-axew-surface select-none">

      {/* Tool Group */}
      <div className="flex items-center gap-0.5 bg-axew-panel rounded px-1
py-0.5 mr-2">
        {tools.map(({ id, icon: Icon, label, shortcut }) => (
          <ToolButton
            key={id}
            active={activeTool === id}
            onClick={() => setActiveTool(id)}
            title={`${label} (${shortcut})`}
          >
            <Icon size={14} />
          </ToolButton>
        ))}
      </div>
    </div>
  )
}
```

```

</div>

<Divider />

{ /* Undo / Redo */ }
<ToolButton onClick={() => undo()} disabled={!canUndo} title="Undo
(Ctrl+Z)">
  <Undo2 size={14} />
</ToolButton>
<ToolButton onClick={() => redo()} disabled={!canRedo} title="Redo
(Ctrl+Shift+Z)">
  <Redo2 size={14} />
</ToolButton>

<Divider />

{ /* Playback Controls */ }
<ToolButton onClick={previousFrame} title="Previous Frame (,)">
  <ChevronLeft size={14} />
</ToolButton>
<ToolButton onClick={stop} title="Stop">
  <Square size={13} />
</ToolButton>
<motion.button
  whileTap={{ scale: 0.92 }}
  className={clsx(
    'flex items-center justify-center w-7 h-7 rounded transition-
colors',
    playing
      ? 'bg-axew-accent text-white hover:bg-axew-accentHover'
      : 'text-axew-text bg-axew-panel hover:bg-axew-borderLight'
    )}
  onClick={togglePlay}
  title="Play/Pause (Space)"
>
  {playing ? <Pause size={13} /> : <Play size={13} />}
</motion.button>
<ToolButton onClick={nextFrame} title="Next Frame (.)">
  <ChevronRight size={14} />
</ToolButton>

<Divider />

{ /* Snap & Zoom */ }
<ToolButton onClick={toggleSnap} active={snapEnabled} title="Toggle
Snap (N)">
  <Magnet size={14} />
</ToolButton>

<ToolButton onClick={() => setZoom(Math.max(10, zoom - 20))}
title="Zoom Out (-)">
  <ZoomOut size={14} />
</ToolButton>

<div className="text-2xs text-axew-textMuted font-mono px-1 min-w-
[44px] text-center">
  {zoom}px/s
</div>

```

```

      <ToolButton onClick={() => setZoom(Math.min(1000, zoom + 20))}
title="Zoom In (+)">
      <ZoomIn size={14} />
    </ToolButton>

    { /* Spacer */ }
    <div className="flex-1" />

    { /* AI Panel Toggle */ }
    <ToolButton
      onClick={() => togglePanel('ai')}
      active={layout.aiPanelOpen}
      title="AI Assistant"
      className="text-axew-ai"
    >
      <Sparkles size={14} />
    </ToolButton>

    <Divider />

    { /* Import / Export */ }
    <ToolButton title="Import Media (Ctrl+I)">
      <Upload size={14} />
    </ToolButton>

    <motion.button
      whileTap={{ scale: 0.95 }}
      className="flex items-center gap-1.5 h-7 px-3 rounded bg-axew-
accent text-white text-xs font-medium hover:bg-axew-accentHover transition-
colors"
      onClick={() => setShowExportDialog(true)}
      title="Export (Ctrl+E)"
    >
      <Download size={12} />
      Export
    </motion.button>
  </div>
)
}

function ToolButton({
  children,
  active = false,
  disabled = false,
  onClick,
  title,
  className,
}): {
  children: React.ReactNode
  active?: boolean
  disabled?: boolean
  onClick?: () => void
  title?: string
  className?: string
}) {
  return (
    <motion.button
      whileTap={!disabled ? { scale: 0.92 } : undefined}
      className={clsx(

```

```

        'flex items-center justify-center w-7 h-7 rounded transition-
colors',
        active && 'bg-axew-accentSubtle text-axew-accent',
        !active && !disabled && 'text-axew-textMuted hover:text-axew-text
hover:bg-axew-panel',
        disabled && 'text-axew-textDim cursor-not-allowed opacity-50',
        className
    )}
    onClick={!disabled ? onClick : undefined}
    title={title}
    disabled={disabled}
  >
    {children}
  </motion.button>
)
}

function Divider() {
  return <div className="h-4 w-px bg-axew-border mx-1 flex-shrink-0" />
}

```

PHASE 2 — PREVIEW MONITOR

Create `apps/desktop/src/components/preview/PreviewMonitor.tsx`:

tsx

```

import React, { useRef, useEffect, useCallback } from 'react'
import { motion, AnimatePresence } from 'framer-motion'
import { usePlaybackStore } from '../../../stores/playbackStore'
import { useProjectStore } from '../../../stores/projectStore'
import { useTimelineStore } from '../../../stores/timelineStore'
import { formatTimecode } from '../../../lib/timecode'
import { Play, Pause, Volume2, VolumeX, Maximize2, AlertCircle, Loader2 }
from 'lucide-react'
import { VolumeSlider } from './VolumeSlider'
import { clsx } from 'clsx'

export function PreviewMonitor() {
  const videoRef = useRef<HTMLVideoElement>(null)
  const containerRef = useRef<HTMLDivElement>(null)
  const [showControls, setShowControls] = React.useState(false)
  const [controlsTimeout, setControlsTimeout] =
    React.useState<NodeJS.Timeout | null>(null)

  const {
    playing, currentTime, duration, volume, muted, buffering, frameRate,
    togglePlay, setVideoRef, setCurrentTime, setDuration, setFrameRate,
    setBuffering, setPreviewReady, setVolume, setMuted
  } = usePlaybackStore()

  const { currentProject } = useProjectStore()
  const { selectedClipIds } = useTimelineStore()

  // Find the clip to preview
  const previewUrl = React.useMemo(() => {
    if (!currentProject) return null
    const timeline = currentProject.timeline

```

```

    for (const track of timeline.tracks) {
      for (const clip of track.clips) {
        if (currentTime >= clip.startTime && currentTime < clip.startTime + clip.duration) {
          const media = currentProject.mediaFiles[clip.mediaId]
          if (media) return `file://${media.path}`
        }
      }
    }
    return null
  }, [currentProject, currentTime])

  useEffect(() => {
    if (videoRef.current) {
      setVideoRef(videoRef.current)
    }
    return () => setVideoRef(null)
  }, [setVideoRef])

  const handleMouseMove = useCallback(() => {
    setShowControls(true)
    if (controlsTimeout) clearTimeout(controlsTimeout)
    const t = setTimeout(() => setShowControls(false), 2500)
    setControlsTimeout(t)
  }, [controlsTimeout])

  const handleVideoTimeUpdate = useCallback(() => {
    if (videoRef.current) {
      setCurrentTime(videoRef.current.currentTime)
    }
  }, [setCurrentTime])

  const handleVideoLoaded = useCallback(() => {
    if (videoRef.current) {
      setDuration(videoRef.current.duration)
      setPreviewReady(true)
      setBuffering(false)
    }
  }, [setDuration, setPreviewReady, setBuffering])

  const handleVideoWaiting = useCallback(() => setBuffering(true), [setBuffering])
  const handleVideoPlaying = useCallback(() => setBuffering(false), [setBuffering])

  const handleSeekClick = useCallback((e: React.MouseEvent<HTMLDivElement>) => {
    const rect = e.currentTarget.getBoundingClientRect()
    const ratio = (e.clientX - rect.left) / rect.width
    setCurrentTime(ratio * duration)
  }, [duration, setCurrentTime])

  const progressPercent = duration > 0 ? (currentTime / duration) * 100 : 0

  return (
    <div
      ref={containerRef}
      className="relative flex flex-col w-full h-full bg-black overflow-hidden"
    >

```

```

        onMouseMove={handleMouseMove}
        onMouseLeave={() => setShowControls(false)}
      >
        {/* Video Element */}
        <div className="flex-1 flex items-center justify-center relative">
          {previewUrl ? (
            <video
              ref={videoRef}
              src={previewUrl}
              className="max-w-full max-h-full object-contain"
              onUpdate={handleVideoTimeUpdate}
              onLoadedMetadata={handleVideoLoaded}
              onWaiting={handleVideoWaiting}
              onPlaying={handleVideoPlaying}
              onClick={togglePlay}
              style={{ cursor: 'pointer' }}
              muted={muted}
            />
          ) : (
            <EmptyPreview />
          )}

          {/* Buffering Overlay */}
          <AnimatePresence>
            {buffering && (
              <motion.div
                initial={{ opacity: 0 }}
                animate={{ opacity: 1 }}
                exit={{ opacity: 0 }}
                className="absolute inset-0 flex items-center justify-center
bg-black/40"
              >
                <Loader2 size={32} className="text-axew-accent animate-spin"
              />
            )}
          </motion.div>
        </AnimatePresence>

        {/* Play/Pause Center Indicator */}
        <AnimatePresence>
          {showControls && !buffering && previewUrl && (
            <motion.button
              initial={{ opacity: 0, scale: 0.8 }}
              animate={{ opacity: 0.7, scale: 1 }}
              exit={{ opacity: 0, scale: 0.8 }}
              transition={{ duration: 0.15 }}
              className="absolute inset-0 flex items-center justify-center"
              onClick={togglePlay}
            >
              <div className="w-12 h-12 rounded-full bg-black/60 flex
items-center justify-center">
                {playing
                  ? <Pause size={20} className="text-white" />
                  : <Play size={20} className="text-white ml-0.5" />
                }
              </div>
            </motion.button>
          )}
        </AnimatePresence>

```

```

</div>

    { /* Controls Bar */ }
    <AnimatePresence>
      {(showControls || !playing) && (
        <motion.div
          initial={{ opacity: 0, y: 4 }}
          animate={{ opacity: 1, y: 0 }}
          exit={{ opacity: 0, y: 4 }}
          transition={{ duration: 0.15 }}
          className="flex-shrink-0 px-3 pb-2 pt-1 bg-gradient-to-t from-
black/80 to-transparent"
        >
          { /* Progress Bar */ }
          <div
            className="w-full h-1 bg-white/20 rounded-full mb-2 cursor-
pointer group"
            onClick={handleSeekClick}
          >
            <div
              className="h-full bg-axew-accent rounded-full relative
group-hover:h-1.5 transition-all"
              style={{ width: `${progressPercent}%` }}
            >
              <div className="absolute right-0 top-1/2 -translate-y-1/2
w-3 h-3 bg-white rounded-full shadow opacity-0 group-hover:opacity-100
transition-opacity" />
            </div>
          </div>

          { /* Controls Row */ }
          <div className="flex items-center gap-2">
            <button
              className="text-white/70 hover:text-white transition-
colors"
              onClick={togglePlay}
            >
              {playing ? <Pause size={15} /> : <Play size={15} />}
            </button>

            <div className="text-white/60 text-2xs font-mono">
              {formatTimecode(currentTime, frameRate)}
              <span className="mx-1"></span>
              {formatTimecode(duration, frameRate)}
            </div>

            <div className="flex-1" />

            <VolumeSlider volume={volume} muted={muted}
onVolumeChange={setVolume} onMuteToggle={() => setMuted(!muted)} />

            <button className="text-white/70 hover:text-white transition-
colors" title="Fullscreen">
              <Maximize2 size={13} />
            </button>
          </div>
        </motion.div>
      )}
    </AnimatePresence>

```

```

    </div>
  )
}

function EmptyPreview() {
  return (
    <div className="flex flex-col items-center justify-center gap-3 text-axew-textDim">
      <div className="w-16 h-16 rounded-full bg-axew-panel flex items-center justify-center">
        <Play size={24} className="text-axew-textDim ml-0.5" />
      </div>
      <div className="text-center">
        <p className="text-sm text-axew-textMuted">No preview</p>
        <p className="text-xs mt-1">Add clips to the timeline to begin</p>
      </div>
    </div>
  )
}

```

Create `apps/desktop/src/components/preview/VolumeSlider.tsx`:

tsx

```

import React from 'react'
import { Volume2, VolumeX, Volume1 } from 'lucide-react'

interface VolumeSliderProps {
  volume: number
  muted: boolean
  onVolumeChange: (volume: number) => void
  onMuteToggle: () => void
}

export function VolumeSlider({ volume, muted, onVolumeChange, onMuteToggle }: VolumeSliderProps) {
  const [showSlider, setShowSlider] = React.useState(false)

  const VolumeIcon = muted || volume === 0 ? VolumeX : volume < 0.5 ? Volume1 : Volume2

  return (
    <div
      className="flex items-center gap-1.5 relative"
      onMouseEnter={() => setShowSlider(true)}
      onMouseLeave={() => setShowSlider(false)}
    >
      <button
        className="text-white/70 hover:text-white transition-colors"
        onClick={onMuteToggle}
      >
        <VolumeIcon size={14} />
      </button>

      {showSlider && (
        <input
          type="range"
          min="0"
          max="1"
          step="0.01"
          value={muted ? 0 : volume}

```

```

        onChange={ (e) => onVolumeChange(parseFloat(e.target.value)) }
        className="w-16 h-1 accent-axew-accent cursor-pointer"
      />
    )}
  </div>
)
}

```

PHASE 2 — MEDIA BIN PANEL

Create `apps/desktop/src/components/panels/MediaBinPanel.tsx`:

tsx

```

import React, { useCallback, useState } from 'react'
import { motion, AnimatePresence } from 'framer-motion'
import { useProjectStore } from '../../../stores/projectStore'
import { useUIStore } from '../../../stores/uiStore'
import { importMediaFiles } from '../../../lib/mediaImport'
import { FolderOpen, Grid3X3, List, Search, Plus, Film, Music, Image, X }
from 'lucide-react'
import { clsx } from 'clsx'
import type { MediaFile } from '@shared/media'

type ViewMode = 'grid' | 'list'

export function MediaBinPanel() {
  const { currentProject, addMediaFile } = useProjectStore()
  const { setIsMediaImporting, addNotification } = useUIStore()
  const [viewMode, setViewMode] = useState<ViewMode>('grid')
  const [searchQuery, setSearchQuery] = useState('')
  const [isDraggingOver, setIsDraggingOver] = useState(false)
  const [selectedMedia, setSelectedMedia] = useState<string | null>(null)

  const mediaFiles = currentProject
    ? Object.values(currentProject.mediaFiles)
    : []

  const filtered = mediaFiles.filter(f =>
    f.name.toLowerCase().includes(searchQuery.toLowerCase())
  )

  const handleImportClick = useCallback(async () => {
    const result = await window.axew.dialog.openFile({
      title: 'Import Media',
      filters: [
        { name: 'Video', extensions: ['mp4', 'mov', 'mkv', 'avi', 'webm',
'mxf', 'r3d'] },
        { name: 'Audio', extensions: ['mp3', 'wav', 'aac', 'flac', 'ogg',
'm4a'] },
        { name: 'Image', extensions: ['jpg', 'jpeg', 'png', 'tiff', 'bmp',
'dpx', 'exr'] },
        { name: 'All Media', extensions: ['mp4', 'mov', 'mkv', 'avi',
'webm', 'mp3', 'wav', 'aac', 'jpg', 'png'] },
      ],
      properties: ['openFile', 'multiSelections'],
    })
  })

```

```

    if (result.canceled || !result.filePaths.length) return

    setIsMediaImporting(true)
    try {
      const imported = await importMediaFiles(result.filePaths)
      for (const media of imported) {
        addMediaFile(media)
      }
      addNotification({
        type: 'success',
        message: `Imported ${imported.length} file${imported.length !== 1 ?
's' : ''}`,
      })
    } catch (err) {
      addNotification({ type: 'error', message: `Import failed:
${String(err)} ` })
    } finally {
      setIsMediaImporting(false)
    }
  }, [addMediaFile, addNotification, setIsMediaImporting])

  const handleDrop = useCallback(async (e: React.DragEvent) => {
    e.preventDefault()
    setIsDraggingOver(false)
    const files = Array.from(e.dataTransfer.files).map(f =>
f.path).filter(Boolean)
    if (!files.length) return

    setIsMediaImporting(true)
    try {
      const imported = await importMediaFiles(files)
      for (const media of imported) {
        addMediaFile(media)
      }
      addNotification({ type: 'success', message: `Imported
${imported.length} file(s) ` })
    } catch (err) {
      addNotification({ type: 'error', message: `Import failed:
${String(err)} ` })
    } finally {
      setIsMediaImporting(false)
    }
  }, [addMediaFile, addNotification, setIsMediaImporting])

  return (
    <div
      className={clsx(
        'flex flex-col h-full bg-axew-surface transition-colors',
        isDraggingOver && 'bg-axew-accentSubtle'
      )}
      onDragOver={(e) => { e.preventDefault(); setIsDraggingOver(true) }}
      onDragLeave={() => setIsDraggingOver(false)}
      onDrop={handleDrop}
    >
      { /* Header */ }
      <div className="flex items-center justify-between px-3 py-2 border-b
border-axew-border flex-shrink-0">
        <span className="text-xs font-medium text-axew-textMuted uppercase
tracking-wider">Media Bin</span>

```

```

        <div className="flex items-center gap-1">
            <button
                className={clsx('p-1 rounded hover:bg-axew-panel transition-
colors', viewMode === 'grid' && 'text-axew-accent')}>
                onClick={() => setViewMode('grid')}
            >
                <Grid3X3 size={12} />
            </button>
            <button
                className={clsx('p-1 rounded hover:bg-axew-panel transition-
colors', viewMode === 'list' && 'text-axew-accent')}>
                onClick={() => setViewMode('list')}
            >
                <List size={12} />
            </button>
            <button
                className="p-1 rounded hover:bg-axew-panel transition-colors
text-axew-textMuted hover:text-axew-text"
                onClick={handleImportClick}
                title="Import Media"
            >
                <Plus size={12} />
            </button>
        </div>
    </div>

    { /* Search */ }
    <div className="px-2 py-1.5 border-b border-axew-border flex-shrink-
0">
        <div className="flex items-center gap-1.5 bg-axew-panel rounded px-
2 py-1">
            <Search size={10} className="text-axew-textDim flex-shrink-0" />
            <input
                type="text"
                placeholder="Search media..."
                value={searchQuery}
                onChange={(e) => setSearchQuery(e.target.value)}
                className="flex-1 bg-transparent text-2xs text-axew-text
placeholder-axew-textDim outline-none"
            />
            {searchQuery && (
                <button onClick={() => setSearchQuery('')} className="text-
axew-textDim hover:text-axew-textMuted">
                    <X size={10} />
                </button>
            )}
        </div>
    </div>

    { /* Media List */ }
    <div className="flex-1 overflow-y-auto p-2">
        {filtered.length === 0 ? (
            <EmptyMediaBin onImport={handleImportClick}
isDraggingOver={isDraggingOver} />
        ) : viewMode === 'grid' ? (
            <MediaGrid files={filtered} selectedId={selectedMedia}
onSelect={setSelectedMedia} />
        ) : (

```

```

        <MediaList files={filtered} selectedId={selectedMedia}
onSelect={setSelectedMedia} />
    )}
</div>

    { /* Stats */
    {mediaFiles.length > 0 && (
        <div className="px-3 py-1.5 border-t border-axew-border flex-
shrink-0">
            <span className="text-2xs text-axew-textDim">{mediaFiles.length}
item{mediaFiles.length !== 1 ? 's' : ''}</span>
        </div>
    )}
</div>
)
}

function EmptyMediaBin({ onImport, isDraggingOver }: { onImport: () =>
void; isDraggingOver: boolean }) {
    return (
        <motion.div
            animate={{ opacity: isDraggingOver ? 0.9 : 0.6 }}
            className="flex flex-col items-center justify-center h-40 gap-3 text-
axew-textDim cursor-pointer"
            onClick={onImport}
        >
            <div className={clsx(
                'w-12 h-12 rounded-xl border-2 border-dashed flex items-center
justify-center transition-colors',
                isDraggingOver ? 'border-axew-accent text-axew-accent' : 'border-
axew-border'
            )}>
                <FolderOpen size={20} />
            </div>
            <div className="text-center">
                <p className="text-xs">{isDraggingOver ? 'Drop to import' : 'Click
to import'}</p>
                <p className="text-2xs mt-0.5">or drag files here</p>
            </div>
        </motion.div>
    )
}

function MediaGrid({ files, selectedId, onSelect }: { files: MediaFile[];
selectedId: string | null; onSelect: (id: string) => void }) {
    return (
        <div className="grid grid-cols-2 gap-1.5">
            {files.map((file) => (
                <MediaGridItem key={file.id} file={file} selected={selectedId ===
file.id} onSelect={() => onSelect(file.id)} />
            ))}
        </div>
    )
}

function MediaGridItem({ file, selected, onSelect }: { file: MediaFile;
selected: boolean; onSelect: () => void }) {
    const icon = file.type === 'video' ? Film : file.type === 'audio' ? Music
: Image

```

```

    return (
      <motion.div
        whileHover={{ scale: 1.02 }}
        whileTap={{ scale: 0.98 }}
        className={clsx(
          'relative rounded overflow-hidden border cursor-pointer transition-
colors',
          selected ? 'border-axew-accent bg-axew-accentSubtle' : 'border-
axew-border bg-axew-panel hover:border-axew-borderLight'
        )}
        onClick={onSelect}
        draggable
        onDragStart={(e) => {
          e.dataTransfer.setData('application/axew-media', JSON.stringify({
mediaId: file.id })))
        }}
      >
        <div className="aspect-video bg-axew-timeline flex items-center
justify-center overflow-hidden">
          {file.thumbnail ? (
            <img src={`file://${file.thumbnail}`} alt={file.name}
className="w-full h-full object-cover" />
          ) : (
            <div className="text-axew-textDim">
              {React.createElement(icon, { size: 20 })}
            </div>
          )}
          <div className="absolute bottom-0.5 right-0.5 bg-black/70 text-
white text-2xs px-1 py-0.5 rounded font-mono">
            {formatDuration(file.duration)}
          </div>
          <div>
            <div className="px-1.5 py-1">
              <p className="text-2xs text-axew-text truncate"
title={file.name}>{file.name}</p>
            </div>
          </div>
        </motion.div>
      )
    )
  }

function MediaList({ files, selectedId, onSelect }: { files: MediaFile[];
selectedId: string | null; onSelect: (id: string) => void }) {
  return (
    <div className="flex flex-col gap-0.5">
      {files.map((file) => (
        <MediaListItem key={file.id} file={file} selected={selectedId ===
file.id} onSelect={() => onSelect(file.id)} />
      ))}
    </div>
  )
}

function MediaListItem({ file, selected, onSelect }: { file: MediaFile;
selected: boolean; onSelect: () => void }) {
  const Icon = file.type === 'video' ? Film : file.type === 'audio' ? Music
: Image

  return (

```

```

    <div
      className={clsx(
        'flex items-center gap-2 px-2 py-1.5 rounded cursor-pointer
transition-colors',
        selected ? 'bg-axew-accentSubtle text-axew-accent' : 'hover:bg-
axew-panel text-axew-text'
      )}
      onClick={onSelect}
      draggable
      onDragStart={(e) => {
        e.dataTransfer.setData('application/axew-media', JSON.stringify({
mediaId: file.id })))
      }}
    >
      <Icon size={12} className="flex-shrink-0 text-axew-textMuted" />
      <span className="flex-1 text-2xs truncate">{file.name}</span>
      <span className="text-2xs text-axew-textDim font-
mono">{formatDuration(file.duration)}</span>
    </div>
  )
}

function formatDuration(seconds: number): string {
  if (!seconds || isNaN(seconds)) return '--:--'
  const m = Math.floor(seconds / 60)
  const s = Math.floor(seconds % 60)
  return `${m}:${s.toString().padStart(2, '0')}`
}

```

PHASE 2 — TIMELINE COMPONENT

Create `apps/desktop/src/components/timeline/Timeline.tsx`:

tsx

```

import React, { useRef, useCallback, useEffect, useState } from 'react'
import { useProjectStore } from '../../../stores/projectStore'
import { useTimelineStore } from '../../../stores/timelineStore'
import { usePlaybackStore } from '../../../stores/playbackStore'
import { TimelineRuler } from './TimelineRuler'
import { TrackHeader } from './TrackHeader'
import { TrackLane } from './TrackLane'
import { Playhead } from './Playhead'
import { TimelineToolbar } from './TimelineToolbar'
import { clsx } from 'clsx'

const TRACK_HEADER_WIDTH = 180

export function Timeline() {
  const { currentProject } = useProjectStore()
  const { zoom, scrollX, scrollY, setScrollX, setScrollY } =
useTimelineStore()
  const { currentTime, duration, setCurrentTime } = usePlaybackStore()
  const scrollContainerRef = useRef<HTMLDivElement>(null)
  const timelineRef = useRef<HTMLDivElement>(null)
  const [timelineWidth, setTimelineWidth] = useState(0)

  const timeline = currentProject?.timeline

```

```

const tracks = timeline?.tracks ?? []
const totalDuration = Math.max(duration, timeline?.duration ?? 0, 30)
const totalWidth = totalDuration * zoom

useEffect(() => {
  const observer = new ResizeObserver((entries) => {
    for (const entry of entries) {
      setTimelineWidth(entry.contentRect.width - TRACK_HEADER_WIDTH)
    }
  })
  if (timelineRef.current) observer.observe(timelineRef.current)
  return () => observer.disconnect()
}, [])

const handleRulerClick = useCallback((e:
React.MouseEvent<HTMLDivElement>) => {
  const rect = e.currentTarget.getBoundingClientRect()
  const x = e.clientX - rect.left + scrollX
  const time = x / zoom
  setCurrentTime(Math.max(0, Math.min(totalDuration, time)))
}, [scrollX, zoom, totalDuration, setCurrentTime])

const handleWheel = useCallback((e: React.WheelEvent) => {
  if (e.ctrlKey || e.metaKey) {
    e.preventDefault()
    const factor = e.deltaY > 0 ? 0.85 : 1.18
    const { setZoom } = useTimelineStore.getState()
    setZoom(zoom * factor)
  } else if (e.shiftKey) {
    e.preventDefault()
    setScrollX(scrollX + e.deltaY)
  } else {
    setScrollY(scrollY + e.deltaY)
  }
}, [zoom, scrollX, scrollY, setScrollX, setScrollY])

const handleScroll = useCallback((e: React.UIEvent<HTMLDivElement>) => {
  setScrollX(e.currentTarget.scrollLeft)
}, [setScrollX])

if (!currentProject) {
  return (
    <div className="flex items-center justify-center w-full h-full bg-
axew-timeline text-axew-textDim text-xs">
      No project open – create a project to begin editing
    </div>
  )
}

const playheadX = currentTime * zoom - scrollX

return (
  <div
    ref={timelineRef}
    className="flex flex-col w-full h-full bg-axew-timeline overflow-
hidden"
    onWheel={handleWheel}
  >
    <TimelineToolbar />

```

```

        {/** Main Timeline Area **/}
        <div className="flex flex-1 overflow-hidden">
            {/** Track Headers Column **/}
            <div
                className="flex flex-col flex-shrink-0 border-r border-axew-
border overflow-hidden"
                style={{ width: TRACK_HEADER_WIDTH }}
            >
                {/** Header spacer for ruler **/}
                <div className="h-6 border-b border-axew-border bg-axew-surface
flex-shrink-0" />

                {/** Track headers - scroll vertically with tracks **/}
                <div
                    className="flex-1 overflow-hidden"
                    style={{ transform: `translateY(-${scrollY}px)` }}
                >
                    {tracks.map((track) => {
                        <TrackHeader key={track.id} track={track} />
                    })}
                </div>
            </div>

            {/** Scrollable Timeline Area **/}
            <div
                className="flex-1 flex flex-col overflow-hidden relative"
            >
                {/** Ruler **/}
                <div
                    className="relative flex-shrink-0 h-6 border-b border-axew-
border overflow-hidden cursor-pointer"
                    onClick={handleRulerClick}
                >
                    <TimelineRuler
                        zoom={zoom}
                        scrollX={scrollX}
                        duration={totalDuration}
                        width={timelineWidth}
                    />
                    {/** Playhead on ruler **/}
                    <div
                        className="absolute top-0 bottom-0 w-px bg-axew-playhead
pointer-events-none z-10"
                        style={{ left: playheadX }}
                    />
                </div>

                {/** Tracks Container **/}
                <div
                    ref={scrollContainerRef}
                    className="flex-1 overflow-x-scroll overflow-y-hidden relative"
                    style={{ scrollbarWidth: 'thin' }}
                    onScroll={handleScroll}
                >
                    {/** Playhead Line **/}
                    <div
                        className="absolute top-0 bottom-0 w-px bg-axew-playhead z-20
pointer-events-none"

```

```

        style={{ left: currentTime * zoom }}
      />
    { /* Track Lanes */}
    <div
      className="relative"
      style={{
        width: totalWidth + timelineWidth,
        transform: `translateY(-${scrollY}px)`,
      }}
    >
      {tracks.map((track) => (
        <TrackLane
          key={track.id}
          track={track}
          zoom={zoom}
          scrollX={scrollX}
        />
      ))}

      {tracks.length === 0 && (
        <div className="flex items-center justify-center h-20 text-
        axew-textDim text-xs">
          Add tracks to start editing
        </div>
      )}
    </div>

    { /* Drop zone for adding clips */}
  </div>
</div>
</div>
</div>
)
}

```

Create `apps/desktop/src/components/timeline/TimelineRuler.tsx`:

tsx

```

import React, { useMemo } from 'react'
import { formatTimecode } from '../../lib/timecode'

interface TimelineRulerProps {
  zoom: number
  scrollX: number
  duration: number
  width: number
  frameRate?: number
}

export function TimelineRuler({ zoom, scrollX, duration, width, frameRate =
30 }: TimelineRulerProps) {
  const markers = useMemo(() => {
    const result: { time: number; major: boolean; label: string }[] = []
    if (width <= 0 || zoom <= 0) return result

    const visibleStart = scrollX / zoom
    const visibleEnd = visibleStart + width / zoom

    // Determine tick interval based on zoom

```

```

    let interval: number
    if (zoom > 400) interval = 1 / frameRate // every frame
    else if (zoom > 100) interval = 1 // every second
    else if (zoom > 50) interval = 2
    else if (zoom > 20) interval = 5
    else if (zoom > 10) interval = 10
    else if (zoom > 5) interval = 30
    else interval = 60

    const majorInterval = interval * 5
    const start = Math.floor(visibleStart / interval) * interval

    for (let t = start; t <= Math.min(duration, visibleEnd + interval); t
+= interval) {
        const major = Math.abs(t % majorInterval) < 0.001
        result.push({
            time: t,
            major,
            label: major ? formatTimecode(t, frameRate, false) : '',
        })
    }
    return result
}, [zoom, scrollX, duration, width, frameRate])

return (
    <div className="w-full h-full relative bg-axew-surface overflow-
hidden">
        <svg
            className="absolute inset-0 w-full h-full"
            style={{ overflow: 'visible' }}
        >
            {markers.map(({ time, major, label }) => {
                const x = time * zoom - scrollX
                if (x < -20 || x > width + 20) return null
                return (
                    <g key={time}>
                        <line
                            x1={x} y1={major ? 0 : 12}
                            x2={x} y2={24}
                            stroke={major ? '#3E3E4E' : '#22222A'}
                            strokeWidth={major ? 1 : 0.5}
                        />
                        {label && (
                            <text
                                x={x + 2}
                                y={10}
                                fill="#4B4B5E"
                                fontSize="9"
                                fontFamily="JetBrains Mono, monospace"
                            >
                                {label}
                            </text>
                        )}
                    </g>
                )
            })}
        </svg>
    </div>
)

```

```
}
```

Create `apps/desktop/src/components/timeline/TrackHeader.tsx`:

tsx

```
import React from 'react'
import { Eye, EyeOff, Volume2, VolumeX, Lock, Unlock, Film, Music } from
' lucide-react '
import { useTimelineStore } from '../../stores/timelineStore'
import type { Track } from '@shared/timeline'
import { clsx } from 'clsx'

interface TrackHeaderProps {
  track: Track
}

export function TrackHeader({ track }: TrackHeaderProps) {
  const { updateTrack } = useTimelineStore()

  const isVideo = track.type === 'video'
  const isAudio = track.type === 'audio'

  return (
    <div
      className={clsx(
        'flex items-center gap-1.5 px-2 border-b border-axew-border bg-
axew-surface select-none',
        'hover:bg-axew-panel transition-colors'
      )}
      style={{ height: track.height }}
    >
      { /* Track Type Icon */ }
      <div className="text-axew-textDim flex-shrink-0">
        {isVideo ? <Film size={11} /> : isAudio ? <Music size={11} /> :
null}
      </div>

      { /* Track Name */ }
      <span className="flex-1 text-2xs text-axew-textMuted truncate"
title={track.name}>
        {track.name}
      </span>

      { /* Controls */ }
      <div className="flex items-center gap-0.5 flex-shrink-0">
        {isVideo && (
          <button
            className={clsx(
              'p-0.5 rounded transition-colors',
              track.visible ? 'text-axew-textDim hover:text-axew-text' :
'text-axew-textDim opacity-40'
            )}
            onClick={() => updateTrack(track.id, { visible: !track.visible
})}
            title={track.visible ? 'Hide' : 'Show'}
          >
            {track.visible ? <Eye size={11} /> : <EyeOff size={11} />}
          </button>
        )}
        {isAudio && (
```

```

        <button
          className={clsx(
            'p-0.5 rounded transition-colors',
            !track.muted ? 'text-axew-textDim hover:text-axew-text' :
            'text-axew-textDim opacity-40'
          )}
          onClick={() => updateTrack(track.id, { muted: !track.muted })}
          title={track.muted ? 'Unmute' : 'Mute'}
        >
          {track.muted ? <VolumeX size={11} /> : <Volume2 size={11} />}
        </button>
      )}
    <button
      className={clsx(
        'p-0.5 rounded transition-colors',
        !track.locked ? 'text-axew-textDim hover:text-axew-text' :
        'text-axew-accent'
      )}
      onClick={() => updateTrack(track.id, { locked: !track.locked })}
      title={track.locked ? 'Unlock' : 'Lock'}
    >
      {track.locked ? <Lock size={11} /> : <Unlock size={11} />}
    </button>
  </div>
</div>
)
}

```

Create `apps/desktop/src/components/timeline/TrackLane.tsx`:

tsx

```

import React, { useCallback } from 'react'
import { useTimelineStore } from '../../../stores/timelineStore'
import { useProjectStore } from '../../../stores/projectStore'
import { ClipItem } from './ClipItem'
import type { Track } from '@shared/timeline'
import { clsx } from 'clsx'

interface TrackLaneProps {
  track: Track
  zoom: number
  scrollX: number
}

export function TrackLane({ track, zoom, scrollX }: TrackLaneProps) {
  const { addClip, snapEnabled } = useTimelineStore()
  const { currentProject } = useProjectStore()

  const handleDrop = useCallback((e: React.DragEvent<HTMLDivElement>) => {
    e.preventDefault()
    const data = e.dataTransfer.getData('application/axew-media')
    if (!data) return

    try {
      const { mediaId } = JSON.parse(data)
      const media = currentProject?.mediaFiles[mediaId]
      if (!media) return

      const rect = e.currentTarget.getBoundingClientRect()
      const x = e.clientX - rect.left + scrollX
    }
  })

```

```

        let startTime = x / zoom

        // Snap to integer frames at common frame rates
        if (snapEnabled) {
            const fps = currentProject?.timeline.frameRate ?? 30
            startTime = Math.round(startTime * fps) / fps
        }

        addClip(track.id, {
            mediaId,
            name: media.name,
            startTime: Math.max(0, startTime),
            duration: media.duration,
            mediaInPoint: 0,
            mediaOutPoint: media.duration,
            speed: 1,
            opacity: 1,
            volume: 1,
            disabled: false,
            color: null,
            effects: [],
            transitions: { in: null, out: null },
            keyframes: [],
        })
    } catch { /* ignore */ }
}, [addClip, currentProject, scrollX, track.id, zoom, snapEnabled])

return (
    <div
        className={clsx(
            'relative border-b border-axew-border overflow-hidden select-none',
            track.locked && 'opacity-60 pointer-events-none',
            !track.visible && 'opacity-30',
            track.type === 'video' ? 'bg-axew-timeline' : 'bg-[#0C0C10]'
        )}
        style={{ height: track.height }}
        onDragOver={(e) => e.preventDefault()}
        onDrop={handleDrop}
    >
        {/* Track background grid lines */}
        <div className="absolute inset-0 opacity-10" style={{
            backgroundImage: `repeating-linear-gradient(90deg, transparent,
transparent ${zoom - 1}px, #444 ${zoom}px)`,
            backgroundSize: `${zoom}px 100%`,
            backgroundPositionX: -scrollX % zoom,
        }} />

        {/* Clips */}
        {track.clips.map((clip) => (
            <ClipItem
                key={clip.id}
                clip={clip}
                track={track}
                zoom={zoom}
                scrollX={scrollX}
            />
        ))}
    </div>
)

```

```
}]
```

Create `apps/desktop/src/components/timeline/ClipItem.tsx`:

tsx

```
import React, { useCallback, useState, useRef } from 'react'
import { motion } from 'framer-motion'
import { useTimelineStore } from '../../../stores/timelineStore'
import { useProjectStore } from '../../../stores/projectStore'
import type { Clip, Track } from '@shared/timeline'
import { clsx } from 'clsx'

interface ClipItemProps {
  clip: Clip
  track: Track
  zoom: number
  scrollX: number
}

const HANDLE_WIDTH = 6

export function ClipItem({ clip, track, zoom, scrollX }: ClipItemProps) {
  const { selectedClipIds, selectClip, moveClip, trimClipIn, trimClipOut,
splitClip, deleteClip } = useTimelineStore()
  const { currentProject } = useProjectStore()
  const isSelected = selectedClipIds.includes(clip.id)

  const [isDragging, setIsDragging] = useState(false)
  const [isTrimming, setIsTrimming] = useState<'in' | 'out' | null>(null)
  const dragStartRef = useRef({ x: 0, startTime: 0 })

  const left = clip.startTime * zoom
  const width = clip.duration * zoom
  const visibleLeft = left - scrollX

  const media = currentProject?.mediaFiles[clip.mediaId]

  const handleClick = useCallback((e: React.MouseEvent) => {
    e.stopPropagation()
    selectClip(clip.id, e.metaKey || e.ctrlKey)
  }, [clip.id, selectClip])

  const handleDragStart = useCallback((e: React.DragEvent) => {
    e.stopPropagation()
    setIsDragging(true)
    dragStartRef.current = { x: e.clientX, startTime: clip.startTime }
    e.dataTransfer.setData('application/axew-clip', JSON.stringify({
      clipId: clip.id,
      trackId: track.id,
      startTime: clip.startTime,
    }))
    e.dataTransfer.effectAllowed = 'move'
  }, [clip.id, clip.startTime, track.id])

  const handleDragEnd = useCallback(() => {
    setIsDragging(false)
  }, [])

  const handleTrimMouseDown = useCallback((e: React.MouseEvent, side: 'in'
| 'out') => {
```

```

    e.stopPropagation()
    e.preventDefault()
    setIsTrimming(side)

    const startX = e.clientX
    let lastX = startX

    const onMove = (me: MouseEvent) => {
      const delta = (me.clientX - lastX) / zoom
      lastX = me.clientX
      if (side === 'in') {
        trimClipIn(clip.id, delta)
      } else {
        trimClipOut(clip.id, delta)
      }
    }

    const onUp = () => {
      setIsTrimming(null)
      document.removeEventListener('mousemove', onMove)
      document.removeEventListener('mouseup', onUp)
    }

    document.addEventListener('mousemove', onMove)
    document.addEventListener('mouseup', onUp)
  }, [clip.id, zoom, trimClipIn, trimClipOut])

  const handleDoubleClick = useCallback((e: React.MouseEvent) => {
    e.stopPropagation()
    // Split at click position
    const rect = e.currentTarget.getBoundingClientRect()
    const x = e.clientX - rect.left
    const time = clip.startTime + x / zoom
    splitClip(clip.id, time)
  }, [clip.id, clip.startTime, zoom, splitClip])

  const handleKeyDown = useCallback((e: React.KeyboardEvent) => {
    if (isSelected && (e.key === 'Delete' || e.key === 'Backspace')) {
      e.preventDefault()
      deleteClip(clip.id)
    }
  }, [clip.id, deleteClip, isSelected])

  if (visibleLeft + width < 0 || visibleLeft > 2000) return null

  return (
    <motion.div
      className={clsx(
        'absolute top-1 bottom-1 rounded overflow-hidden cursor-pointer
group',
        'border clip-shadow transition-colors',
        isSelected
          ? 'border-axew-accent ring-1 ring-axew-accent/50 z-10'
          : 'border-axew-clipBorder hover:border-axew-accent/50',
        isDragging && 'opacity-60',
        isTrimming && 'cursor-col-resize',
        track.type === 'video' ? 'bg-axew-clip' : 'bg-[#1A3A2F]',
      )}
      style={{

```

```

        left: visibleLeft,
        width: Math.max(4, width),
    }}
    onClick={handleClick}
    onDoubleClick={handleDoubleClick}
    draggable
    onDragStart={handleDragStart}
    onDragEnd={handleDragEnd}
    tabIndex={0}
    onKeyDown={handleKeyDown}
  >
    {/* Thumbnail strip for video clips */}
    {track.type === 'video' && media?.thumbnail && (
      <div className="absolute inset-0 opacity-30">
        <img
          src={`file://${media.thumbnail}`}
          alt=""
          className="w-full h-full object-cover object-left"
          style={{ imageRendering: 'crisp-edges' }}
        />
      </div>
    )}

    {/* Waveform for audio clips */}
    {track.type === 'audio' && (
      <div className="absolute inset-0 flex items-center px-1">
        <div className="w-full h-6 opacity-40" style={{
          background: 'repeating-linear-gradient(90deg, #3D6B8A 0px,
#3D6B8A 1px, transparent 1px, transparent 3px)',
        }} />
      </div>
    )}

    {/* Clip Label */}
    <div className="absolute top-0 left-0 right-0 flex items-center px-
1.5 pt-0.5">
      <span className="text-2xs text-white/80 truncate font-medium
leading-tight drop-shadow">
        {clip.name}
      </span>
    </div>

    {/* Trim Handle - In */}
    <div
      className={clsx(
        'absolute left-0 top-0 bottom-0 cursor-col-resize z-20',
        'flex items-center justify-center',
        'opacity-0 group-hover:opacity-100 hover:opacity-100 transition-
opacity',
        'bg-white/20 hover:bg-axew-accent/60'
      )}
      style={{ width: HANDLE_WIDTH }}
      onMouseDown={(e) => handleTrimMouseDown(e, 'in')}
    />

    {/* Trim Handle - Out */}
    <div
      className={clsx(
        'absolute right-0 top-0 bottom-0 cursor-col-resize z-20',

```

```

        'flex items-center justify-center',
        'opacity-0 group-hover:opacity-100 hover:opacity-100 transition-
opacity',
        'bg-white/20 hover:bg-axew-accent/60'
    )}
    style={{ width: HANDLE_WIDTH }}
    onMouseDown={ (e) => handleTrimMouseDown(e, 'out') }
  />

  { /* Selected indicator */ }
  { isSelected && (
    <div className="absolute inset-0 ring-1 ring-inset ring-axew-
accent/50 pointer-events-none" />
  ) }
</motion.div>
)
}

```

Create `apps/desktop/src/components/timeline/TimelineToolbar.tsx`:

`tsx`

```

import React from 'react'
import { Plus, Trash2, ChevronDown, Film, Music, Type } from 'lucide-react'
import { useTimelineStore } from '../../../stores/timelineStore'
import type { TrackType } from '@shared/timeline'
import { clsx } from 'clsx'

export function TimelineToolbar() {
  const { addTrack, deleteClip, selectedClipIds } = useTimelineStore()

  const handleAddTrack = (type: TrackType) => {
    addTrack(type)
  }

  const handleDeleteSelected = () => {
    for (const id of selectedClipIds) {
      deleteClip(id)
    }
  }

  return (
    <div className="flex items-center h-7 px-2 gap-1 border-b border-axew-
border bg-axew-surface flex-shrink-0">
      <span className="text-2xs text-axew-textDim uppercase tracking-wider
mr-1">Tracks</span>

      <button
        className="flex items-center gap-1 text-axew-textMuted hover:text-
axew-text text-2xs px-1.5 py-0.5 rounded hover:bg-axew-panel transition-
colors"
        onClick={() => handleAddTrack('video')}
        title="Add Video Track"
      >
        <Plus size={10} />
        <Film size={9} />
        <span>Video</span>
      </button>

      <button

```

```

        className="flex items-center gap-1 text-axew-textMuted hover:text-axew-text text-2xs px-1.5 py-0.5 rounded hover:bg-axew-panel transition-colors"
        onClick={() => handleAddTrack('audio')}
        title="Add Audio Track"
      >
        <Plus size={10} />
        <Music size={9} />
        <span>Audio</span>
      </button>

      <button
        className="flex items-center gap-1 text-axew-textMuted hover:text-axew-text text-2xs px-1.5 py-0.5 rounded hover:bg-axew-panel transition-colors"
        onClick={() => handleAddTrack('subtitle')}
        title="Add Subtitle Track"
      >
        <Plus size={10} />
        <Type size={9} />
        <span>Sub</span>
      </button>

    <div className="flex-1" />

    {selectedClipIds.length > 0 && {
      <button
        className="flex items-center gap-1 text-axew-error hover:text-red-400 text-2xs px-1.5 py-0.5 rounded hover:bg-red-950 transition-colors"
        onClick={handleDeleteSelected}
        title="Delete Selected (Delete)"
      >
        <Trash2 size={10} />
        <span>Delete ({selectedClipIds.length})</span>
      </button>
    )}
  </div>
)
}

```

Create `apps/desktop/src/components/timeline/Playhead.tsx`:

tsx

```

import React from 'react'

interface PlayheadProps {
  x: number
  height: number
}

export function Playhead({ x, height }: PlayheadProps) {
  return (
    <div
      className="absolute top-0 pointer-events-none z-30"
      style={{ left: x, height }}
    >
      </* Head */>
      <div className="absolute -top-0.5 left-1/2 -translate-x-1/2 w-3 h-3 bg-axew-playhead" style={{
        clipPath: 'polygon(50% 100%, 0 0, 100% 0)'
      }}
    >

```

```

    }} />
    { /* Line */}
    <div className="absolute top-3 left-1/2 -translate-x-1/2 w-px bg-
axew-playhead h-full" />
    </div>
  )
}

```

PHASE 2 — AI PANEL

Create `apps/desktop/src/components/panels/AIPanel.tsx`:

tsx

```

import React, { useState, useRef, useEffect } from 'react'
import { motion, AnimatePresence } from 'framer-motion'
import { Sparkles, Send, Loader2, ChevronDown, Scissors, Wand2, FileText,
Brain } from 'lucide-react'
import { useAIStore } from '../../../stores/aiStore'
import { useProjectStore } from '../../../stores/projectStore'
import { useTimelineStore } from '../../../stores/timelineStore'
import { clsx } from 'clsx'
import { sendAIMessage, detectSilence, generateTranscript } from
'../../../lib/aiClient'
import type { AIMessage } from '@shared/ai'

const QUICK ACTIONS = [
  { id: 'cut-silence', icon: Scissors, label: 'Cut Silence', description:
'Remove silent parts automatically' },
  { id: 'transcribe', icon: FileText, label: 'Transcribe', description:
'Generate transcript & subtitles' },
  { id: 'highlight', icon: Wand2, label: 'Highlights', description: 'Find
best moments' },
  { id: 'suggest', icon: Brain, label: 'Story Edit', description: 'AI story
suggestions' },
]

export function AIPanel() {
  const { messages, isThinking, streamingContent, activeModel,
aiServiceStatus, addMessage, setIsThinking, appendStreamingContent,
finalizeStreamingMessage } = useAIStore()
  const { currentProject } = useProjectStore()
  const [input, setInput] = useState('')
  const messagesEndRef = useRef<HTMLDivElement>(null)
  const inputRef = useRef<HTMLTextAreaElement>(null)

  useEffect(() => {
    messagesEndRef.current?.scrollIntoView({ behavior: 'smooth' })
  }, [messages, streamingContent])

  const handleSend = async () => {
    const text = input.trim()
    if (!text || isThinking) return

    addMessage({ role: 'user', content: text })
    setInput('')
    setIsThinking(true)
  }

```

```

    try {
      const timeline = currentProject?.timeline ?? null
      await sendAIMessage(text, timeline, {
        onChunk: appendStreamingContent,
        onDone: finalizeStreamingMessage,
        onError: (err) => {
          finalizeStreamingMessage()
          addMessage({ role: 'assistant', content: `Error: ${err}` })
        },
      })
    } catch (err) {
      setIsThinking(false)
      addMessage({ role: 'assistant', content: `Failed to get response: ${err}` })
    }
  }

  const handleKeyDown = (e: React.KeyboardEvent) => {
    if (e.key === 'Enter' && !e.shiftKey) {
      e.preventDefault()
      handleSend()
    }
  }

  const handleQuickAction = async (actionId: string) => {
    if (!currentProject) return
    switch (actionId) {
      case 'cut-silence':
        addMessage({ role: 'user', content: 'Detect and remove silences from my video' })
        setIsThinking(true)
        // Will be implemented in Phase 6
        setTimeout(() => {
          addMessage({ role: 'assistant', content: 'Silence detection requires the AI service to be running. Start the AI service to use this feature.' })
          setIsThinking(false)
        }, 500)
        break
      case 'transcribe':
        addMessage({ role: 'user', content: 'Generate transcript for my media' })
        setIsThinking(true)
        setTimeout(() => {
          addMessage({ role: 'assistant', content: 'Transcription requires Whisper model. Install it via: `ollama pull whisper` or configure the AI service.' })
          setIsThinking(false)
        }, 500)
        break
      default:
        addMessage({ role: 'user', content: `Run ${actionId}` })
    }
  }

  return (
    <div className="flex flex-col h-full bg-axew-surface">
      {/* Header */}
    </div>
  )

```



```

      ? 'bg-axew-ai text-white hover:bg-axew-aiGlow'
      : 'bg-axew-border text-axew-textDim cursor-not-allowed'
    )}
    onClick={handleSend}
    disabled={!input.trim() || isThinking}
  >
    {isThinking ? <Loader2 size={11} className="animate-spin" /> :
<Send size={11} />}
  </motion.button>
</div>
<p className="text-2xs text-axew-textDim mt-1 text-center">Enter to
send · Shift+Enter for newline</p>
</div>
</div>
)
}

function MessageBubble({ message }: { message: AIMessage }) {
  const isUser = message.role === 'user'
  return (
    <div className={clsx('flex gap-2', isUser && 'flex-row-reverse')}>
      <div className={clsx(
        'w-5 h-5 rounded-full flex items-center justify-center flex-shrink-
0 mt-0.5 text-2xs font-bold',
        isUser ? 'bg-axew-accentSubtle text-axew-accent' : 'bg-axew-ai/20
text-axew-ai'
      )}>
        {isUser ? 'U' : <Sparkles size={10} />}
      </div>
      <div className={clsx(
        'flex-1 rounded-lg p-2 text-xs',
        isUser ? 'bg-axew-accentSubtle text-axew-text' : 'bg-axew-panel
text-axew-text'
      )}>
        <p className="whitespace-pre-wrap leading-
relaxed">{message.content}</p>
      </div>
    </div>
  )
}

```

PHASE 2 — INSPECTOR PANEL

Create `apps/desktop/src/components/panels/InspectorPanel.tsx`:

tsx

```

import React from 'react'
import { useTimelineStore } from '../../../stores/timelineStore'
import { useProjectStore } from '../../../stores/projectStore'
import { ClipInspector } from '../inspector/ClipInspector'
import { MediaInspector } from '../inspector/MediaInspector'
import { Settings2 } from 'lucide-react'

export function InspectorPanel() {
  const { selectedClipIds } = useTimelineStore()
  const { currentProject } = useProjectStore()

```

```

const selectedClip = React.useMemo(() => {
  if (!currentProject || selectedClipIds.length !== 1) return null
  for (const track of currentProject.timeline.tracks) {
    const clip = track.clips.find(c => c.id === selectedClipIds[0])
    if (clip) return clip
  }
  return null
}, [currentProject, selectedClipIds])

return (
  <div className="flex flex-col h-full bg-axew-surface">
    <div className="flex items-center justify-between px-3 py-2 border-b border-axew-border flex-shrink-0">
      <span className="text-xs font-medium text-axew-textMuted uppercase tracking-wider">Inspector</span>
      <Settings2 size={12} className="text-axew-textDim" />
    </div>

    <div className="flex-1 overflow-y-auto">
      {selectedClip ? (
        <ClipInspector clip={selectedClip} />
      ) : (
        <EmptyInspector />
      )}
    </div>
  </div>
)
}

function EmptyInspector() {
  return (
    <div className="flex flex-col items-center justify-center h-40 gap-2 text-axew-textDim">
      <Settings2 size={20} className="opacity-30" />
      <p className="text-xs">Select a clip to inspect</p>
    </div>
  )
}

```

Create `apps/desktop/src/components/panels/inspector/ClipInspector.tsx`:

tsx

```

import React from 'react'
import { useTimelineStore } from '../../../stores/timelineStore'
import { useProjectStore } from '../../../stores/projectStore'
import type { Clip } from '@shared/timeline'
import { formatTimecode } from '../../../lib/timecode'

interface ClipInspectorProps {
  clip: Clip
}

export function ClipInspector({ clip }: ClipInspectorProps) {
  const { updateClip } = useTimelineStore()
  const { currentProject } = useProjectStore()
  const media = currentProject?.mediaFiles[clip.mediaId]

  return (
    <div className="p-3 space-y-4 text-xs">
      {/* Clip Name */}

```

```

    <Section title="Clip">
      <input
        type="text"
        value={clip.name}
        onChange={(e) => updateClip(clip.id, { name: e.target.value })}
        className="w-full bg-axew-panel border border-axew-border rounded
px-2 py-1 text-axew-text text-xs outline-none focus:border-axew-accent"
      />
    </Section>

    { /* Timing */ }
    <Section title="Timing">
      <InfoRow label="Start" value={formatTimecode(clip.startTime, 30)} />
      <InfoRow label="Duration" value={formatTimecode(clip.duration, 30)} />
      <InfoRow label="In Point" value={formatTimecode(clip.mediaInPoint,
30)} />
      <InfoRow label="Out Point"
value={formatTimecode(clip.mediaOutPoint, 30)} />
    </Section>

    { /* Video */ }
    {media?.type === 'video' && (
      <Section title="Video">
        <InspectorSlider
          label="Opacity"
          value={clip.opacity * 100}
          min={0}
          max={100}
          step={1}
          unit="%"
          onChange={(v) => updateClip(clip.id, { opacity: v / 100 })}
        />
        <InspectorSlider
          label="Speed"
          value={clip.speed * 100}
          min={10}
          max={400}
          step={5}
          unit="%"
          onChange={(v) => updateClip(clip.id, { speed: v / 100 })}
        />
      </Section>
    ) }

    { /* Audio */ }
    <Section title="Audio">
      <InspectorSlider
        label="Volume"
        value={clip.volume * 100}
        min={0}
        max={200}
        step={1}
        unit="%"
        onChange={(v) => updateClip(clip.id, { volume: v / 100 })}
      />
    </Section>

```

```

        { /* Source Media */ }
        {media && (
            <Section title="Source">
                <InfoRow label="File" value={media.name} />
                <InfoRow label="Type" value={media.type} />
                {media.width > 0 && <InfoRow label="Size"
value={` ${media.width}x${media.height}`} />}
                <InfoRow label="Codec" value={media.codec || 'unknown'} />
            </Section>
        )}
    </div>
)
}

function Section({ title, children }: { title: string; children:
React.ReactNode }) {
    return (
        <div>
            <h3 className="text-2xs text-axew-textDim uppercase tracking-wider
mb-2">{title}</h3>
            <div className="space-y-1.5">{children}</div>
        </div>
    )
}

function InfoRow({ label, value }: { label: string; value: string }) {
    return (
        <div className="flex items-center justify-between">
            <span className="text-axew-textMuted text-2xs">{label}</span>
            <span className="text-axew-text text-2xs font-mono">{value}</span>
        </div>
    )
}

function InspectorSlider({ label, value, min, max, step, unit, onChange }:
{
    label: string
    value: number
    min: number
    max: number
    step: number
    unit: string
    onChange: (v: number) => void
}) {
    return (
        <div className="flex items-center gap-2">
            <span className="text-axew-textMuted text-2xs w-14 flex-shrink-
0">{label}</span>
            <input
                type="range"
                min={min}
                max={max}
                step={step}
                value={value}
                onChange={(e) => onChange(Number(e.target.value))}
                className="flex-1 h-1 accent-axew-accent cursor-pointer"
            />
            <span className="text-axew-text text-2xs font-mono w-12 text-
right">{Math.round(value)}{unit}</span>

```

```
    </div>
  )
}
```

PHASE 2 — WELCOME SCREEN

Create `apps/desktop/src/components/welcome/WelcomeScreen.tsx`:

tsx

```
import React, { useState } from 'react'
import { motion } from 'framer-motion'
import { Zap, Plus, FolderOpen, Film, Settings } from 'lucide-react'
import { useProjectStore } from '../../../stores/projectStore'
import { useUIStore } from '../../../stores/uiStore'

interface WelcomeScreenProps {
  onClose: () => void
}

export function WelcomeScreen({ onClose }: WelcomeScreenProps) {
  const { createProject, loadProject, recentProjects } = useProjectStore()
  const { setShowWelcome } = useUIStore()
  const [projectName, setProjectName] = useState('Untitled Project')
  const [isCreating, setIsCreating] = useState(false)

  const handleCreate = () => {
    createProject(projectName)
    setShowWelcome(false)
    onClose()
  }

  const handleOpen = async () => {
    const result = await window.axew.dialog.openFile({
      title: 'Open Project',
      filters: [{ name: 'AXEW Project', extensions: ['.axew'] }],
      properties: ['openFile'],
    })
    if (result.canceled || !result.filePaths.length) return
    await loadProject(result.filePaths[0])
    setShowWelcome(false)
    onClose()
  }

  return (
    <div className="flex flex-col items-center justify-center w-full h-full
bg-axew-bg relative overflow-hidden">
      {/* Background grid */}
      <div className="absolute inset-0 opacity-5" style={{
        backgroundImage: 'linear-gradient(#5B5BFF 1px, transparent 1px),
linear-gradient(90deg, #5B5BFF 1px, transparent 1px)',
        backgroundSize: '60px 60px',
      }} />

      {/* Glow */}
      <div className="absolute top-1/2 left-1/2 -translate-x-1/2 -
translate-y-1/2 w-96 h-96 bg-axew-accent/10 rounded-full blur-3xl pointer-
events-none" />
    </div>
  )
}
```

```

<motion.div
  initial={{ opacity: 0, y: 20 }}
  animate={{ opacity: 1, y: 0 }}
  transition={{ duration: 0.4 }}
  className="relative z-10 flex flex-col items-center gap-8 max-w-lg
w-full px-8"
>
  {/* Logo */}
  <div className="flex flex-col items-center gap-3">
    <div className="w-16 h-16 rounded-2xl bg-axew-accent/20 border
border-axew-accent/30 flex items-center justify-center shadow-glow">
      <Zap size={32} className="text-axew-accent" />
    </div>
    <div className="text-center">
      <h1 className="text-4xl font-bold text-gradient-accent
tracking-tight">AXEW</h1>
      <p className="text-sm text-axew-textMuted mt-1">AI-Native Video
Editor</p>
    </div>
  </div>

  {/* New Project Form */}
  <div className="w-full space-y-3">
    <div>
      <label className="text-xs text-axew-textMuted block mb-
1.5">Project Name</label>
      <input
        type="text"
        value={projectName}
        onChange={(e) => setProjectName(e.target.value)}
        onKeyDown={(e) => e.key === 'Enter' && handleCreate()}
        className="w-full bg-axew-panel border border-axew-border
rounded-md px-3 py-2 text-axew-text text-sm outline-none focus:border-axew-
accent transition-colors"
        placeholder="My Project"
        autoFocus
      />
    </div>

    <motion.button
      whileHover={{ scale: 1.01 }}
      whileTap={{ scale: 0.99 }}
      className="w-full flex items-center justify-center gap-2 py-2.5
rounded-md bg-axew-accent hover:bg-axew-accentHover text-white text-sm
font-medium transition-colors"
      onClick={handleCreate}
    >
      <Plus size={16} />
      Create New Project
    </motion.button>

    <button
      className="w-full flex items-center justify-center gap-2 py-2.5
rounded-md bg-axew-panel border border-axew-border hover:border-axew-
borderLight text-axew-textMuted hover:text-axew-text text-sm transition-
colors"
      onClick={handleOpen}
    >

```

```

        <FolderOpen size={15} />
        Open Project
      </button>
    </div>

    { /* Recent Projects */ }
    { recentProjects.length > 0 && (
      <div className="w-full">
        <h3 className="text-xs text-axew-textDim uppercase tracking-
wider mb-2">Recent</h3>
        <div className="space-y-1">
          { recentProjects.slice(0, 5).map((rp) => (
            <button
              key={rp.id}
              className="w-full flex items-center gap-2 px-2.5 py-2
rounded bg-axew-panel/50 hover:bg-axew-panel border border-transparent
hover:border-axew-border text-left transition-colors"
              onClick={() => loadProject(rp.path).then(() => {
setShowWelcome(false); onClose() })}
            >
              <Film size={13} className="text-axew-textDim flex-shrink-
0" />
              <span className="flex-1 text-sm text-axew-textMuted
truncate">{rp.name}</span>
              <span className="text-2xs text-axew-textDim">{new
Date(rp.lastOpenedAt).toLocaleDateString()}</span>
            </button>
          )))
        </div>
      </div>
    ) }

    <p className="text-2xs text-axew-textDim">
      v0.1.0 · Built with Electron + Rust + Python
    </p>
  </motion.div>
</div>
)
}

```

PHASE 2 — NOTIFICATION STACK

Create `apps/desktop/src/components/ui/NotificationStack.tsx`:

tsx

```

import React from 'react'
import { AnimatePresence, motion } from 'framer-motion'
import { useUIStore } from '../../../stores/uiStore'
import { X, CheckCircle, AlertCircle, AlertTriangle, Info } from 'lucide-
react'
import { clsx } from 'clsx'

const icons = {
  success: CheckCircle,
  error: AlertCircle,
  warning: AlertTriangle,
  info: Info,

```

```

    ]

const colors = {
  success: 'border-axew-success/30 bg-green-950/80',
  error: 'border-axew-error/30 bg-red-950/80',
  warning: 'border-axew-warning/30 bg-amber-950/80',
  info: 'border-axew-accent/30 bg-axew-accentSubtle/80',
}

const iconColors = {
  success: 'text-axew-success',
  error: 'text-axew-error',
  warning: 'text-axew-warning',
  info: 'text-axew-accent',
}

export function NotificationStack() {
  const { notifications, removeNotification } = useUIStore()

  return (
    <div className="fixed bottom-8 right-4 z-50 flex flex-col gap-2
pointer-events-none">
      <AnimatePresence>
        {notifications.map((n) => {
          const Icon = icons[n.type]
          return (
            <motion.div
              key={n.id}
              initial={{ opacity: 0, x: 40, scale: 0.95 }}
              animate={{ opacity: 1, x: 0, scale: 1 }}
              exit={{ opacity: 0, x: 40, scale: 0.95 }}
              transition={{ duration: 0.2, ease: 'easeOut' }}
              className={clsx(
                'flex items-center gap-2.5 px-3 py-2.5 rounded-lg border
backdrop-blur-sm shadow-panel pointer-events-auto',
                'max-w-sm',
                colors[n.type]
              )}>
                <Icon size={14} className={clsx('flex-shrink-0',
iconColors[n.type])} />
                <span className="text-xs text-axew-text flex-
1">{n.message}</span>
                <button
                  className="text-axew-textDim hover:text-axew-text
transition-colors flex-shrink-0"
                  onClick={() => removeNotification(n.id)}
                >
                  <X size={12} />
                </button>
              </motion.div>
            )
          )}
        </AnimatePresence>
      </div>
    )
  }
}

```

PHASE 2 — UTILITY LIBRARIES

Create `apps/desktop/src/lib/timecode.ts`:

typescript

```
export function formatTimecode({
  seconds: number,
  fps: number = 30,
  includeFrames: boolean = true
}): string {
  if (isNaN(seconds) || seconds < 0) seconds = 0
  const h = Math.floor(seconds / 3600)
  const m = Math.floor((seconds % 3600) / 60)
  const s = Math.floor(seconds % 60)
  const f = Math.floor((seconds % 1) * fps)

  if (h > 0) {
    return includeFrames
      ? `${h}:${pad(m)}:${pad(s)}:${pad(f)}`
      : `${h}:${pad(m)}:${pad(s)}`
  }
  return includeFrames
    ? `${pad(m)}:${pad(s)}:${pad(f)}`
    : `${pad(m)}:${pad(s)}`
}

function pad(n: number): string {
  return n.toString().padStart(2, '0')
}

export function timecodeToSeconds(timecode: string, fps: number = 30):
number {
  const parts = timecode.split(':').map(Number)
  if (parts.some(isNaN)) return 0
  if (parts.length === 4) {
    const [h, m, s, f] = parts
    return h * 3600 + m * 60 + s + f / fps
  }
  if (parts.length === 3) {
    const [m, s, f] = parts
    return m * 60 + s + f / fps
  }
  if (parts.length === 2) {
    const [m, s] = parts
    return m * 60 + s
  }
  return parts[0] ?? 0
}
```

Create `apps/desktop/src/lib/projectFactory.ts`:

typescript

```
import { nanoid } from 'nanoid'
import type { Project, ProjectSettings } from '@shared/project'
import type { Timeline } from '@shared/timeline'

export function createDefaultProject(name: string, settingsOverride?:
Partial<ProjectSettings>): Project {
  const id = nanoid()
```

```

const timelineId = nanoid()
const now = Date.now()

const settings: ProjectSettings = {
  timeline: {
    frameRate: 30,
    width: 1920,
    height: 1080,
    sampleRate: 48000,
  },
  playback: {
    proxyEnabled: false,
    proxyResolution: 720,
    cacheSize: 4096,
  },
  ai: {
    preferredModel: 'llama3',
    autoTranscribe: false,
    silenceThresholdDb: -40,
    sceneDetectionSensitivity: 0.4,
  },
  export: {
    defaultPreset: 'h264-1080p',
    outputDirectory: '',
  },
  ...settingsOverride,
}

const timeline: Timeline = {
  id: timelineId,
  projectId: id,
  name: 'Timeline 1',
  duration: 300,
  frameRate: settings.timeline.frameRate,
  width: settings.timeline.width,
  height: settings.timeline.height,
  sampleRate: settings.timeline.sampleRate,
  tracks: [
    {
      id: nanoid(),
      timelineId,
      name: 'Video 1',
      type: 'video',
      index: 0,
      height: 80,
      locked: false,
      muted: false,
      visible: true,
      solo: false,
      color: '#1E3A5F',
      clips: [],
    },
    {
      id: nanoid(),
      timelineId,
      name: 'Audio 1',
      type: 'audio',
      index: 1,
      height: 60,

```

```

        locked: false,
        muted: false,
        visible: true,
        solo: false,
        color: '#1E4F3A',
        clips: [],
      },
    ],
    markers: [],
  }
}

const rootFolderId = nanoid()

return {
  id,
  name,
  description: '',
  path: `${name.replace(/[^a-zA-Z0-9- _]/g, '_').axew}`,
  timeline,
  mediaFiles: {},
  mediaBin: {
    rootFolder: {
      id: rootFolderId,
      name: 'Root',
      parentId: null,
      children: [],
      mediaIds: [],
    },
    folders: {},
  },
  settings,
  createdAt: now,
  updatedAt: now,
  version: '0.1.0',
}
}
}

```

Create `apps/desktop/src/lib/mediaImport.ts`:

typescript

```

import type { MediaFile, MediaType } from '@shared/media'
import { nanoid } from 'nanoid'
import path from 'path-browserify'

const VIDEO_EXTS = new Set(['.mp4', '.mov', '.mkv', '.avi', '.webm',
  '.mxf', '.m4v', '.ts'])
const AUDIO_EXTS = new Set(['.mp3', '.wav', '.aac', '.flac', '.ogg',
  '.m4a', '.opus'])
const IMAGE_EXTS = new Set(['.jpg', '.jpeg', '.png', '.tiff', '.bmp',
  '.dpx', '.exr', '.gif'])

function getMediaType(filePath: string): MediaType {
  const ext = path.extname(filePath).toLowerCase()
  if (VIDEO_EXTS.has(ext)) return 'video'
  if (AUDIO_EXTS.has(ext)) return 'audio'
  if (IMAGE_EXTS.has(ext)) return 'image'
  return 'video'
}

async function probeMedia(filePath: string): Promise<Partial<MediaFile>> {

```

```

    try {
      const response = await fetch(`http://localhost:7001/api/media/probe`, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ path: filePath }),
      })
      if (!response.ok) throw new Error('Probe failed')
      const data = await response.json()
      return {
        duration: data.duration ?? 0,
        width: data.width ?? 0,
        height: data.height ?? 0,
        fps: data.fps ?? 0,
        codec: data.video_codec ?? 'unknown',
        audioCodec: data.audio_codec ?? null,
        sampleRate: data.sample_rate ?? null,
        channels: data.channels ?? null,
        bitrate: data.bitrate ?? 0,
      }
    } catch {
      // Rust service might not be running – return defaults
      return {
        duration: 0,
        width: 0,
        height: 0,
        fps: 30,
        codec: 'unknown',
        audioCodec: null,
        sampleRate: null,
        channels: null,
        bitrate: 0,
      }
    }
  }
}

async function generateThumbnail(filePath: string, mediaType: MediaType):
Promise<string | null> {
  if (mediaType !== 'video') return null
  try {
    const response = await
fetch(`http://localhost:7001/api/media/thumbnail`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ path: filePath, time: 1.0 }),
    })
    if (!response.ok) return null
    const data = await response.json()
    return data.thumbnail_path ?? null
  } catch {
    return null
  }
}

export async function importMediaFiles(filePaths: string[]):
Promise<MediaFile[]> {
  const results: MediaFile[] = []

  for (const filePath of filePaths) {
    const id = nanoid()

```

```

    const type = getMediaType(filePath)
    const name = path.basename(filePath)
    const now = Date.now()

    const [probeData, thumbnail] = await Promise.all([
      probeMedia(filePath),
      generateThumbnail(filePath, type),
    ])

    const mediaFile: MediaFile = {
      id,
      name,
      path: filePath,
      type,
      duration: probeData.duration ?? 0,
      width: probeData.width ?? 0,
      height: probeData.height ?? 0,
      fps: probeData.fps ?? 30,
      codec: probeData.codec ?? 'unknown',
      audioCodec: probeData.audioCodec ?? null,
      sampleRate: probeData.sampleRate ?? null,
      channels: probeData.channels ?? null,
      bitrate: probeData.bitrate ?? 0,
      fileSize: 0,
      thumbnail,
      createdAt: now,
      updatedAt: now,
      metadata: {},
    }

    results.push(mediaFile)
  }

  return results
}

```

Create `apps/desktop/src/lib/aiClient.ts`:

typescript

```

import type { Timeline } from '@shared/timeline'

const AI_SERVICE_URL = 'http://localhost:7002'
const OLLAMA_URL = 'http://localhost:11434'

interface StreamCallbacks {
  onChunk: (chunk: string) => void
  onDone: () => void
  onError: (error: string) => void
}

export async function sendAIMessage(
  message: string,
  timeline: Timeline | null,
  callbacks: StreamCallbacks
): Promise<void> {
  const context = timeline ? buildTimelineContext(timeline) : 'No timeline available'

  const systemPrompt = `You are AXEW, an AI assistant for a professional video editor.`

```

You help editors with timeline operations, story suggestions, pacing, and creative decisions.
Be concise, practical, and action-oriented. When suggesting edits, be specific.

Current Timeline Context:

`${context}``

```
    try {
      // Try AI service first
      const response = await fetch(`${AI_SERVICE_URL}/api/chat/stream`, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ message, system: systemPrompt }),
        signal: AbortSignal.timeout(30000),
      })

      if (response.ok && response.body) {
        const reader = response.body.getReader()
        const decoder = new TextDecoder()
        while (true) {
          const { done, value } = await reader.read()
          if (done) break
          const chunk = decoder.decode(value)
          const lines = chunk.split('\n').filter(Boolean)
          for (const line of lines) {
            if (line.startsWith('data: ')) {
              try {
                const data = JSON.parse(line.slice(6))
                if (data.content) callbacks.onChunk(data.content)
              } catch { /* skip */ }
            }
          }
        }
        callbacks.onDone()
        return
      }
    } catch { /* fall through to Ollama */ }

    // Fallback to Ollama directly
    try {
      const response = await fetch(`${OLLAMA_URL}/api/chat`, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          model: 'llama3',
          messages: [
            { role: 'system', content: systemPrompt },
            { role: 'user', content: message },
          ],
          stream: true,
        }),
        signal: AbortSignal.timeout(60000),
      })

      if (!response.ok || !response.body) {
        throw new Error(`Ollama error: ${response.status}`)
      }
    }
```

```

    const reader = response.body.getReader()
    const decoder = new TextDecoder()
    while (true) {
        const { done, value } = await reader.read()
        if (done) break
        const lines = decoder.decode(value).split('\n').filter(Boolean)
        for (const line of lines) {
            try {
                const data = JSON.parse(line)
                if (data.message?.content)
                    callbacks.onChunk(data.message.content)
                if (data.done) break
            } catch { /* skip */ }
        }
    }
    callbacks.onDone()
} catch (err) {}
callbacks.onError(
    'Could not connect to AI service. Ensure Ollama is running (ollama
serve) or start the AI service.'
)
}
}

function buildTimelineContext(timeline: Timeline): string {
    const tracks = timeline.tracks.map(t => {
        const clips = t.clips.map(c =>
            - `${c.name}` at ${c.startTime.toFixed(2)}s, duration
`${c.duration.toFixed(2)}s`
        ).join('\n')
        return `Track "${t.name}" (${t.type}):${clips} || '\n      (empty)'`
    }).join('\n')

    return `Timeline: ${timeline.name}
Duration: ${timeline.duration}s
Frame Rate: ${timeline.frameRate}fps
Resolution: ${timeline.width}x${timeline.height}
Tracks:
${tracks}`
}

export async function checkOllamaStatus(): Promise<boolean> {
    try {
        const resp = await fetch(`${OLLAMA_URL}/api/tags`, { signal:
AbortSignal.timeout(3000) })
        return resp.ok
    } catch {}
    return false
}

export async function checkAIServiceStatus(): Promise<boolean> {
    try {
        const resp = await fetch(`${AI_SERVICE_URL}/health`, { signal:
AbortSignal.timeout(3000) })
        return resp.ok
    } catch {}
    return false
}

```

```

    ]
  }

  export async function detectSilence(mediaPath: string, thresholdDb: number = -40): Promise<unknown[]> {
    const resp = await fetch(`${AI_SERVICE_URL}/api/analysis/silence`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ media_path: mediaPath, threshold_db: thresholdDb })
    })
    if (!resp.ok) throw new Error('Silence detection failed')
    return resp.json()
  }

  export async function generateTranscript(mediaPath: string): Promise<unknown> {
    const resp = await fetch(`${AI_SERVICE_URL}/api/analysis/transcribe`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ media_path: mediaPath })
    })
    if (!resp.ok) throw new Error('Transcription failed')
    return resp.json()
  }
}

```

PHASE 2 — HOOKS

Create `apps/desktop/src/hooks/useKeyboardShortcuts.ts`:

typescript

```

import { useEffect } from 'react'
import { usePlaybackStore } from '../stores/playbackStore'
import { useTimelineStore } from '../stores/timelineStore'
import { useUIStore } from '../stores/uiStore'

export function useKeyboardShortcuts() {
  const { togglePlay, nextFrame, previousFrame, stop } = usePlaybackStore()
  const { undo, redo, selectedClipIds, deleteClip, setZoom, zoom, toggleSnap } = useTimelineStore()
  const { setShowSettings, setShowExportDialog } = useUIStore()

  useEffect(() => {
    const handle = (e: KeyboardEvent) => {
      const active = document.activeElement
      const isInput = active instanceof HTMLInputElement || active instanceof HTMLTextAreaElement

      if (!isInput) {
        switch (e.key) {
          case ' ':
            e.preventDefault()
            togglePlay()
            break
          case 'k':
            if (!e.ctrlKey && !e.metaKey) { e.preventDefault(); togglePlay() }
            break
        }
      }
    }
  })
}

```

```

        case 'j':
            e.preventDefault()
            previousFrame()
            break
        case 'l':
            e.preventDefault()
            nextFrame()
            break
        case ',':
            e.preventDefault()
            previousFrame()
            break
        case '.':
            e.preventDefault()
            nextFrame()
            break
        case 'Delete':
        case 'Backspace':
            if (selectedClipIds.length > 0) {
                e.preventDefault()
                selectedClipIds.forEach(id => deleteClip(id))
            }
            break
        case 'n':
        case 'N':
            e.preventDefault()
            toggleSnap()
            break
        case '=':
        case '+':
            e.preventDefault()
            setZoom(Math.min(1000, zoom * 1.25))
            break
        case '-':
            e.preventDefault()
            setZoom(Math.max(10, zoom * 0.8))
            break
        case '0':
            e.preventDefault()
            setZoom(100)
            break
    }
}

if (e.ctrlKey || e.metaKey) {
    switch (e.key) {
        case 'z':
            e.preventDefault()
            if (e.shiftKey) redo(); else undo()
            break
        case 'y':
            e.preventDefault()
            redo()
            break
        case ',':
            e.preventDefault()
            setShowSettings(true)
            break
        case 'e':

```

```

        e.preventDefault()
        setShowExportDialog(true)
        break
    }
}
}

window.addEventListener('keydown', handle)
return () => window.removeEventListener('keydown', handle)
}, [togglePlay, nextFrame, previousFrame, stop, undo, redo,
selectedClipIds, deleteClip, setZoom, zoom, toggleSnap, setShowSettings,
setShowExportDialog])
}

```

Create `apps/desktop/src/hooks/useServiceStatus.ts`:

typescript

```

import { useEffect } from 'react'
import { useAIStore } from '../stores/aiStore'
import { checkOllamaStatus, checkAIServiceStatus } from '../lib/aiClient'

export function useServiceStatus() {
    const { setOllamaStatus, setAIServiceStatus } = useAIStore()

    useEffect(() => {
        const check = async () => {
            setOllamaStatus('checking')
            setAIServiceStatus('checking')

            const [ollama, ai] = await Promise.all([
                checkOllamaStatus(),
                checkAIServiceStatus(),
            ])

            setOllamaStatus(ollama ? 'connected' : 'disconnected')
            setAIServiceStatus(ai ? 'connected' : 'disconnected')
        }

        check()
        const interval = setInterval(check, 30000)
        return () => clearInterval(interval)
    }, [setOllamaStatus, setAIServiceStatus])
}

```

Create `apps/desktop/src/hooks/useMenuEvents.ts`:

typescript

```

import { useEffect } from 'react'
import { useProjectStore } from '../stores/projectStore'
import { useUIStore } from '../stores/uiStore'

export function useMenuEvents() {
    const { createProject, loadProject, saveProject } = useProjectStore()
    const { setShowWelcome, setShowExportDialog, setIsMediaImporting } =
    useUIStore()

    useEffect(() => {
        const handlers: Array<[string, () => void]> = [
            ['new-project', () => setShowWelcome(true)],
            ['open-project', async () => {
                const result = await window.axew.dialog.openFile({

```

```

        title: 'Open Project',
        filters: [{ name: 'AXEW Project', extensions: ['axew'] }],
        properties: ['openFile'],
    })
    if (!result.canceled && result.filePaths[0]) {
        await loadProject(result.filePaths[0])
        setShowWelcome(false)
    }
}],
    ['save-project', () => saveProject()],
    ['export', () => setShowExportDialog(true)],
    ['import-media', () => {
        window.axew.menu.on('import-media', () => {})
    }],
]

for (const [event, handler] of handlers) {
    window.axew.menu.on(event, handler as () => void)
}

return () => {
    for (const [event, handler] of handlers) {
        window.axew.menu.off(event, handler as () => void)
    }
}
}, [createProject, loadProject, saveProject, setShowWelcome,
setShowExportDialog])
}

```

PHASE 3 — RUST BACKEND ENGINE

Run: `cd crates && cargo new --bin axew-core`

Create `crates/axew-core/Cargo.toml`:

toml

```

[package]
name = "axew-core"
version = "0.1.0"
edition = "2021"

[[bin]]
name = "axew-core"
path = "src/main.rs"

[dependencies]
tokio = { version = "1.37", features = ["full"] }
axum = { version = "0.7", features = ["multipart", "ws"] }
tower = "0.4"
tower-http = { version = "0.5", features = ["cors", "fs", "trace"] }
serde = { version = "1.0", features = ["derive"] }
serde_json = "1.0"
uuid = { version = "1.8", features = ["v4", "serde"] }
anyhow = "1.0"
tracing = "0.1"
tracing-subscriber = { version = "0.3", features = ["env-filter"] }
rusqlite = { version = "0.31", features = ["bundled"] }

```

```

tokio-rusqlite = "0.5"
request = { version = "0.12", features = ["json", "stream"] }
futures = "0.3"
bytes = "1.6"
mime = "0.3"
base64 = "0.22"
sha2 = "0.10"
hex = "0.4"
chrono = { version = "0.4", features = ["serde"] }
dirs = "5.0"
walkdir = "2.5"
glob = "0.3"

```

```

[dev-dependencies]
tokio-test = "0.4"

```

Create crates/axew-core/src/main.rs:

rust

```

mod api;
mod media;
mod db;
mod config;
mod error;

use std::net::SocketAddr;
use axum::{Router, middleware};
use tower_http::cors::{CorsLayer, Any};
use tower_http::trace::TraceLayer;
use tracing_subscriber::{layer::SubscriberExt, util::SubscriberInitExt};
use crate::config::AppConfig;
use crate::db::Database;

#[tokio::main]
async fn main() -> anyhow::Result<()> {
    tracing_subscriber::registry()
        .with(tracing_subscriber::EnvFilter::new(
            std::env::var("RUST_LOG").unwrap_or_else(|_|
"axew_core=debug,tower_http=info".into()),
        ))
        .with(tracing_subscriber::fmt::layer())
        .init();

    let config = AppConfig::from env();
    let db = Database::new(&config.db_path).await?;

    tracing::info!("Starting AXEW Core v0.1.0 on port {}", config.port);

    let app_state = api::AppState {
        config: config.clone(),
        db: std::sync::Arc::new(db),
    };

    let app = Router::new()
        .nest("/api", api::router(app_state))
        .layer(
            CorsLayer::new()
                .allow_origin(Any)
                .allow_methods(Any)
                .allow_headers(Any)

```

```

    )
    .layer(TraceLayer::new_for_http());

    let addr: SocketAddr = format!("127.0.0.1:{}", config.port).parse()?;
    tracing::info!("AXEW Core listening on {}", addr);

    let listener = tokio::net::TcpListener::bind(addr).await?;
    axum::serve(listener, app).await?;

    Ok(())
}

```

Create `crates/axew-core/src/config.rs`:

rust

```

#[derive(Debug, Clone)]
pub struct AppConfig {
    pub port: u16,
    pub db_path: String,
    pub cache_dir: String,
    pub ffmpeg_path: String,
    pub ffprobe_path: String,
}

impl AppConfig {
    pub fn from_env() -> Self {
        let home = dirs::home_dir()
            .unwrap_or_default()
            .to_string_lossy()
            .to_string();

        Self {
            port: std::env::var("AXEW_PORT")
                .ok()
                .and_then(|p| p.parse().ok())
                .unwrap_or(7001),
            db_path: format!("{}/.axew/axew.db", home),
            cache_dir: std::env::var("AXEW_CACHE_DIR")
                .unwrap_or_else(|_| format!("{}/.axew/cache", home)),
            ffmpeg_path: std::env::var("AXEW_FFMPEG_PATH")
                .unwrap_or_else(|_| "ffmpeg".to_string()),
            ffprobe_path: std::env::var("AXEW_FFPROBE_PATH")
                .unwrap_or_else(|_| "ffprobe".to_string()),
        }
    }
}

```

Create `crates/axew-core/src/error.rs`:

rust

```

use axum::{
    http::StatusCode,
    response::{IntoResponse, Response},
    Json,
};
use serde_json::json;

#[derive(Debug)]
pub enum AppError {
    NotFound(String),
    BadRequest(String),
}

```

```

        Internal(anyhow::Error),
        IoError(std::io::Error),
    }

    impl IntoResponse for AppError {
        fn into_response(self) -> Response {
            let (status, message) = match &self {
                AppError::NotFound(msg) => (StatusCode::NOT_FOUND,
msg.clone()),
                AppError::BadRequest(msg) => (StatusCode::BAD_REQUEST,
msg.clone()),
                AppError::Internal(e) => (StatusCode::INTERNAL_SERVER_ERROR,
e.to_string()),
                AppError::IoError(e) => (StatusCode::INTERNAL_SERVER_ERROR,
e.to_string()),
            };
            (status, Json(json!({ "error": message }))).into_response()
        }
    }

    impl From<anyhow::Error> for AppError {
        fn from(e: anyhow::Error) -> Self { AppError::Internal(e) }
    }

    impl From<std::io::Error> for AppError {
        fn from(e: std::io::Error) -> Self { AppError::IoError(e) }
    }

    pub type Result<T> = std::result::Result<T, AppError>;

```

Create `crates/axew-core/src/db.rs`:

rust

```

use tokio_rusqlite::Connection;
use anyhow::Result;

pub struct Database {
    pub conn: Connection,
}

impl Database {
    pub async fn new(path: &str) -> Result<Self> {
        if let Some(parent) = std::path::Path::new(path).parent() {
            std::fs::create_dir_all(parent)?;
        }

        let conn = Connection::open(path).await?;

        conn.call(|c| {
            c.execute_batch("
                PRAGMA journal_mode=WAL;
                PRAGMA synchronous=NORMAL;
                PRAGMA foreign_keys=ON;
                PRAGMA cache_size=10000;

                CREATE TABLE IF NOT EXISTS media_cache (
                    id TEXT PRIMARY KEY,
                    path TEXT NOT NULL,
                    duration REAL,
                    width INTEGER,

```

```

        height INTEGER,
        fps REAL,
        video_codec TEXT,
        audio_codec TEXT,
        sample_rate INTEGER,
        channels INTEGER,
        bitrate INTEGER,
        thumbnail_path TEXT,
        created_at INTEGER NOT NULL,
        updated_at INTEGER NOT NULL
    );

    CREATE INDEX IF NOT EXISTS idx_media_path ON
media_cache(path);

    CREATE TABLE IF NOT EXISTS thumbnails (
        id TEXT PRIMARY KEY,
        media_path TEXT NOT NULL,
        time_seconds REAL NOT NULL,
        thumbnail_path TEXT NOT NULL,
        created_at INTEGER NOT NULL
    );

    CREATE TABLE IF NOT EXISTS transcripts (
        id TEXT PRIMARY KEY,
        media_path TEXT NOT NULL,
        language TEXT NOT NULL DEFAULT 'en',
        segments_json TEXT NOT NULL,
        full_text TEXT NOT NULL,
        created_at INTEGER NOT NULL
    );

    CREATE TABLE IF NOT EXISTS projects (
        id TEXT PRIMARY KEY,
        name TEXT NOT NULL,
        path TEXT NOT NULL,
        last_opened_at INTEGER NOT NULL,
        created_at INTEGER NOT NULL
    );
    );
    ")?;
    Ok(())
    }).await?;

    Ok(Self { conn })
}
}

Create crates/axew-core/src/media.rs:

```

rust

```

use std::process::Command;
use std::path::Path;
use anyhow::{Result, Context, anyhow};
use serde::{Deserialize, Serialize};

#[derive(Debug, Serialize, Deserialize, Default)]
pub struct MediaProbeResult {
    pub duration: f64,
    pub width: u32,
    pub height: u32,

```

```

    pub fps: f64,
    pub video_codec: Option<String>,
    pub audio_codec: Option<String>,
    pub sample_rate: Option<u32>,
    pub channels: Option<u32>,
    pub bitrate: u64,
    pub has_video: bool,
    pub has_audio: bool,
}

#[derive(Debug, Deserialize)]
struct FFprobeOutput {
    streams: Vec<FFprobeStream>,
    format: FFprobeFormat,
}

#[derive(Debug, Deserialize)]
struct FFprobeStream {
    codec_type: Option<String>,
    codec_name: Option<String>,
    width: Option<u32>,
    height: Option<u32>,
    r_frame_rate: Option<String>,
    avg_frame_rate: Option<String>,
    sample_rate: Option<String>,
    channels: Option<u32>,
    duration: Option<String>,
}

#[derive(Debug, Deserialize)]
struct FFprobeFormat {
    duration: Option<String>,
    bit_rate: Option<String>,
}

pub fn probe_media(path: &str, ffprobe_bin: &str) ->
Result<MediaProbeResult> {
    let output = Command::new(ffprobe_bin)
        .args([
            "-v", "quiet",
            "-print_format", "json",
            "-show_streams",
            "-show_format",
            path,
        ])
        .output()
        .context("Failed to run ffprobe - is it installed and in PATH?")?;

    if !output.status.success() {
        let stderr = String::from_utf8_lossy(&output.stderr);
        return Err(anyhow!("ffprobe failed: {}", stderr));
    }

    let probe: FFprobeOutput = serde_json::from_slice(&output.stdout)
        .context("Failed to parse ffprobe output")?;

    let mut result = MediaProbeResult::default();

    if let Some(dur_str) = &probe.format.duration {

```

```

        result.duration = dur_str.parse().unwrap_or(0.0);
    }

    if let Some(br_str) = &probe.format.bit_rate {
        result.bitrate = br_str.parse::<u64>().unwrap_or(0) / 1000; //
        Convert to kbps
    }

    for stream in &probe.streams {
        match stream.codec_type.as_deref() {
            Some("video") if !result.has_video => {
                result.has_video = true;
                result.video_codec = stream.codec_name.clone();
                result.width = stream.width.unwrap_or(0);
                result.height = stream.height.unwrap_or(0);

                // Parse frame rate from "num/den" format
                let fps_str = stream.avg_frame_rate.as_deref()
                    .or(stream.r_frame_rate.as_deref())
                    .unwrap_or("30/1");
                result.fps = parse_fraction(fps_str).unwrap_or(30.0);

                if result.duration == 0.0 {
                    if let Some(dur_str) = &stream.duration {
                        result.duration = dur_str.parse().unwrap_or(0.0);
                    }
                }
            }
            Some("audio") if !result.has_audio => {
                result.has_audio = true;
                result.audio_codec = stream.codec_name.clone();
                result.sample_rate = stream.sample_rate.as_ref()
                    .and_then(|s| s.parse().ok());
                result.channels = stream.channels;

                if result.duration == 0.0 {
                    if let Some(dur_str) = &stream.duration {
                        result.duration = dur_str.parse().unwrap_or(0.0);
                    }
                }
            }
            _ => {}
        }
    }

    Ok(result)
}

fn parse_fraction(s: &str) -> Option<f64> {
    let parts: Vec<f64> = s.split('/').filter_map(|p|
        p.parse().ok()).collect();
    match parts.as_slice() {
        [num, den] if *den != 0.0 => Some(num / den),
        [num] => Some(*num),
        _ => None,
    }
}

pub fn generate_thumbnail(

```

```

    media_path: &str,
    output_path: &str,
    time_seconds: f64,
    width: u32,
    ffmpeg_bin: &str,
) -> Result<()> {
    if let Some(parent) = Path::new(output_path).parent() {
        std::fs::create_dir_all(parent)?;
    }

    let time_str = format!("{:.3}", time_seconds);
    let scale_filter = format!("scale={}:-2", width);

    let output = Command::new(ffmpeg_bin)
        .args([
            "-ss", &time_str,
            "-i", media_path,
            "-frames:v", "1",
            "-vf", &scale_filter,
            "-q:v", "5",
            "-y",
            output_path,
        ])
        .output()
        .context("Failed to run ffmpeg for thumbnail")?;

    if !output.status.success() {
        let stderr = String::from_utf8_lossy(&output.stderr);
        return Err(anyhow!("ffmpeg thumbnail failed: {}", stderr));
    }

    Ok(())
}

pub fn export_timeline(
    input_path: &str,
    output_path: &str,
    video_codec: &str,
    audio_codec: &str,
    width: u32,
    height: u32,
    frame_rate: f64,
    video_bitrate: u64,
    audio_bitrate: u64,
    crf: u32,
    extra_args: &[String],
    ffmpeg_bin: &str,
    progress_callback: impl Fn(f64) + Send + 'static,
) -> Result<()> {
    let scale_filter = format!("scale={}:{})", width, height);
    let fps_str = format!("{}", frame_rate);
    let crf_str = crf.to_string();
    let video_br = format!("{}", video_bitrate);
    let audio_br = format!("{}", audio_bitrate);

    let mut args = vec![
        "-i", input_path,
        "-vf", &scale_filter,
        "-r", &fps_str,

```

```

        "-c:v", video_codec,
        "-crf", &crf_str,
        "-maxrate", &video_br,
        "-bufsize", "2M",
        "-c:a", audio_codec,
        "-b:a", &audio_br,
        "-y",
        "-progress", "pipe:1",
        "-nostats",
        output_path,
    ];

    let extra: Vec<&str> = extra_args.iter().map(|s| s.as_str()).collect();
    args.extend_from_slice(&extra);

    let mut child = Command::new(ffmpeg_bin)
        .args(&args)
        .stdout(std::process::Stdio::piped())
        .stderr(std::process::Stdio::piped())
        .spawn()?;

    // TODO: parse progress from stdout in Phase 4

    let status = child.wait()?;
    if !status.success() {
        return Err(anyhow!("FFmpeg export failed with status: {}",
status));
    }

    progress_callback(100.0);
    Ok(())
}

```

Create `crates/axew-core/src/api/mod.rs`:

rust

```

pub mod media;
pub mod export;
pub mod health;

use axum::{Router, routing::{get, post}};
use std::sync::Arc;
use crate::{config::AppConfig, db::Database};

#[derive(Clone)]
pub struct AppState {
    pub config: AppConfig,
    pub db: Arc<Database>,
}

pub fn router(state: AppState) -> Router {
    Router::new()
        .route("/health", get(health::health_check))
        .route("/media/probe", post(media::probe_media))
        .route("/media/thumbnail", post(media::generate_thumbnail))
        .route("/media/waveform", post(media::generate_waveform))
        .route("/export/start", post(export::start_export))
        .route("/export/status/:job_id", get(export::get_export_status))
        .route("/export/cancel/:job_id", post(export::cancel_export))
        .with_state(state)
}

```

```
}
```

Create `crates/axew-core/src/api/health.rs`:

rust

```
use axum::Json;
use serde_json::{json, Value};

pub async fn health_check() -> Json<Value> {
    Json(json!({
        "status": "ok",
        "service": "axew-core",
        "version": "0.1.0"
    })))
}
```

```
}
```

Create `crates/axew-core/src/api/media.rs`:

rust

```
use axum::{extract::State, Json};
use serde::{Deserialize, Serialize};
use sha2::{Sha256, Digest};
use std::path::Path;
use crate::api::AppState;
use crate::error::{AppError, Result};
use crate::media as media_ops;

#[derive(Deserialize)]
pub struct ProbeRequest {
    pub path: String,
}

#[derive(Deserialize)]
pub struct ThumbnailRequest {
    pub path: String,
    #[serde(default = "default_time")]
    pub time: f64,
    #[serde(default = "default_width")]
    pub width: u32,
}

fn default_time() -> f64 { 1.0 }
fn default_width() -> u32 { 320 }

pub async fn probe_media(
    State(state): State<AppState>,
    Json(req): Json<ProbeRequest>,
) -> Result<Json<media_ops::MediaProbeResult>> {
    if !Path::new(&req.path).exists() {
        return Err(AppError::NotFound(format!("File not found: {}",
req.path)));
    }

    let path = req.path.clone();
    let ffmpeg = state.config.ffmpeg_path.clone();

    let result = tokio::task::spawn_blocking(move || {
        media_ops::probe_media(&path, &ffmpeg)
    }).await
    .map_err(|e| AppError::Internal(anyhow::anyhow!("Task error: {}", e)))?
    .map_err(AppError::Internal)?;
}
```

```

        Ok(Json(result))
    }

#[derive(Serialize)]
pub struct ThumbnailResponse {
    pub thumbnail_path: String,
    pub width: u32,
    pub height: u32,
}

pub async fn generate_thumbnail(
    State(state): State<AppState>,
    Json(req): Json<ThumbnailRequest>,
) -> Result<Json<ThumbnailResponse>> {
    if !Path::new(&req.path).exists() {
        return Err(AppError::NotFound(format!("File not found: {}",
req.path)));
    }

    // Generate deterministic cache key
    let mut hasher = Sha256::new();
    hasher.update(req.path.as_bytes());
    hasher.update(req.time.to_bits().to_le_bytes());
    hasher.update(req.width.to_le_bytes());
    let hash = hex::encode(hasher.finalize());

    let thumbnail_path = format!("{}",thumbnails/{}.jpg",
state.config.cache_dir, hash);

    if !Path::new(&thumbnail_path).exists() {
        let path = req.path.clone();
        let out = thumbnail_path.clone();
        let ffmpeg = state.config.ffmpeg_path.clone();
        let time = req.time;
        let width = req.width;

        tokio::task::spawn_blocking(move || {
            media_ops::generate_thumbnail(&path, &out, time, width,
&ffmpeg)
        }).await
        .map_err(|e| AppError::Internal( anyhow::anyhow!("Task error: {}",
e)))?
        .map_err(AppError::Internal)?;
    }

    Ok(Json(ThumbnailResponse {
        thumbnail_path,
        width: req.width,
        height: 0,
    })))
}

#[derive(Deserialize)]
pub struct WaveformRequest {
    pub path: String,
    pub samples: Option<u32>,
}

```

```

#[derive(Serialize)]
pub struct WaveformResponse {
    pub samples: Vec<f32>,
    pub duration: f64,
}

pub async fn generate_waveform(
    State(state): State<AppState>,
    Json(req): Json<WaveformRequest>,
) -> Result<Json<WaveformResponse>> {
    if !Path::new(&req.path).exists() {
        return Err(AppError::NotFound(format!("File not found: {}",
req.path)));
    }

    let samples_count = req.samples.unwrap_or(1000);
    let path = req.path.clone();
    let ffmpeg = state.config.ffmpeg_path.clone();

    let result = tokio::task::spawn_blocking(move || ->
anyhow::Result<WaveformResponse> {
        let probe = media_ops::probe_media(&path, &ffmpeg.replace("ffmpeg",
"ffprobe"))?;
        let duration = probe.duration;
        let samples: Vec<f32> = (0..samples_count)
            .map(|i| {
                let t = i as f32 / samples_count as f32;
                (t * 50.0).sin().abs() * 0.8 + (t * 133.0).sin().abs() *
0.2
            })
            .collect();
        Ok(WaveformResponse { samples, duration })
    }).await
    .map_err(|e| AppError::Internal(anyhow::anyhow!("Task error: {}", e)))?
    .map_err(AppError::Internal)?;

    Ok(Json(result))
}

```

Create `crates/axew-core/src/api/export.rs`:

rust

```

use axum::{extract::{State, Path}, Json};
use serde::{Deserialize, Serialize};
use std::collections::HashMap;
use std::sync::{Arc, Mutex};
use crate::api::AppState;
use crate::error::{AppError, Result};

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct ExportJobRequest {
    pub job_id: String,
    pub input_path: String,
    pub output_path: String,
    pub video_codec: String,
    pub audio_codec: String,
    pub width: u32,
    pub height: u32,
    pub frame_rate: f64,
    pub video_bitrate: u64,
}

```

```

    pub audio_bitrate: u64,
    pub crf: u32,
    pub extra_args: Vec<String>,
}

#[derive(Debug, Clone, Serialize)]
pub struct ExportJobStatus {
    pub job_id: String,
    pub status: String,
    pub progress: f64,
    pub error: Option<String>,
}

type JobMap = Arc<Mutex<HashMap<String, ExportJobStatus>>>;

lazy_static::lazy_static! {
    static ref JOBS: JobMap = Arc::new(Mutex::new(HashMap::new()));
}

pub async fn start_export(
    State(state): State<AppState>,
    Json(req): Json<ExportJobRequest>,
) -> Result<Json<ExportJobStatus>> {
    let job_id = req.job_id.clone();

    {
        let mut jobs = JOBS.lock().unwrap();
        jobs.insert(job_id.clone(), ExportJobStatus {
            job_id: job_id.clone(),
            status: "running".to_string(),
            progress: 0.0,
            error: None,
        });
    }

    let jobs_clone = JOBS.clone();
    let job_id_clone = job_id.clone();
    let ffmpeg = state.config.ffmpeg_path.clone();

    tokio::task::spawn_blocking(move || {
        let result = crate::media::export_timeline(
            &req.input_path,
            &req.output_path,
            &req.video_codec,
            &req.audio_codec,
            req.width,
            req.height,
            req.frame_rate,
            req.video_bitrate,
            req.audio_bitrate,
            req.crf,
            &req.extra_args,
            &ffmpeg,
            move |progress| {
                let mut jobs = jobs_clone.lock().unwrap();
                if let Some(job) = jobs.get_mut(&job_id_clone) {
                    job.progress = progress;
                }
            },
        );
    });
}

```

```

    );

    let mut jobs = JOBS.lock().unwrap();
    if let Some(job) = jobs.get_mut(&job_id) {
        match result {
            Ok(_) => {
                job.status = "completed".to_string();
                job.progress = 100.0;
            }
            Err(e) => {
                job.status = "failed".to_string();
                job.error = Some(e.to_string());
            }
        }
    }
});

let jobs = JOBS.lock().unwrap();
let status = jobs.get(&req.job_id).cloned()
    .ok_or_else(|| AppError::NotFound("Job not found".to_string()))?;
Ok(Json(status))
}

pub async fn get_export_status(
    Path(job_id): Path<String>,
) -> Result<Json<ExportJobStatus>> {
    let jobs = JOBS.lock().unwrap();
    let status = jobs.get(&job_id).cloned()
        .ok_or_else(|| AppError::NotFound(format!("Job {} not found",
job_id)))?;
    Ok(Json(status))
}

pub async fn cancel_export(
    Path(job_id): Path<String>,
) -> Result<Json<ExportJobStatus>> {
    let mut jobs = JOBS.lock().unwrap();
    let job = jobs.get_mut(&job_id)
        .ok_or_else(|| AppError::NotFound(format!("Job {} not found",
job_id)))?;
    job.status = "cancelled".to_string();
    Ok(Json(job.clone()))
}

```

Add lazy_static to Cargo.toml dependencies:

toml

```
lazy_static = "1.4"
```

Build the Rust project by running: `cd crates/axew-core && cargo build`

PHASE 5 — PYTHON AI SERVICE

Create `apps/ai-service/requirements.txt`:

```

fastapi>=0.111.0
uvicorn[standard]>=0.29.0
pydantic>=2.7.0
httpx>=0.27.0

```

```
python-multipart>=0.0.9
aiofiles>=23.2.1
numpy>=1.26.0
torch>=2.3.0
torchaudio>=2.3.0
transformers>=4.40.0
openai-whisper>=20231117
onnxruntime>=1.18.0
ffmpeg-python>=0.2.0
librosa>=0.10.2
scipy>=1.13.0
sentence-transformers>=2.7.0
ollama>=0.2.0
```

Create `apps/ai-service/main.py`:

python

```
from fastapi import FastAPI, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import StreamingResponse
import asyncio
import httpx
import logging
import json
import os
from contextlib import asynccontextmanager

from routers import analysis, chat, models
from config import settings

logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(name)s - %(levelname)s - %(message)s')
logger = logging.getLogger(__name__)

@asynccontextmanager
async def lifespan(app: FastAPI):
    logger.info(f"AXEW AI Service starting on port {settings.port}")
    logger.info(f"Models dir: {settings.models_dir}")
    os.makedirs(settings.models_dir, exist_ok=True)
    os.makedirs(settings.cache_dir, exist_ok=True)
    yield
    logger.info("AXEW AI Service shutting down")

app = FastAPI(
    title="AXEW AI Service",
    description="Local AI runtime for AXEW video editor",
    version="0.1.0",
    lifespan=lifespan
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

app.include_router(chat.router, prefix="/api/chat", tags=["chat"])
```

```

app.include_router(analysis.router, prefix="/api/analysis",
tags=["analysis"])
app.include_router(models.router, prefix="/api/models", tags=["models"])

@app.get("/health")
async def health_check():
    return {"status": "ok", "service": "axew-ai", "version": "0.1.0"}

```

Create `apps/ai-service/config.py`:

python

```

from pydantic_settings import BaseSettings
from typing import Optional
import os

class Settings(BaseSettings):
    port: int = 7002
    host: str = "127.0.0.1"
    models_dir: str = os.path.expanduser("~/axew/models")
    cache_dir: str = os.path.expanduser("~/axew/cache")
    ollama_host: str = "http://localhost:11434"
    default_llm_model: str = "llama3"
    default_whisper_model: str = "base"
    device: str = "cpu" # "cuda" or "mps" when available
    log_level: str = "info"

    class Config:
        env_prefix = "AXEW "
        env_file = ".env"

settings = Settings()

```

Create `apps/ai-service/routers/_init_.py`:

Create `apps/ai-service/routers/chat.py`:

python

```

from fastapi import APIRouter, HTTPException
from fastapi.responses import StreamingResponse
from pydantic import BaseModel
from typing import Optional, AsyncGenerator
import httpx
import json
import asyncio
import logging
from config import settings

router = APIRouter()
logger = logging.getLogger(__name__)

class ChatRequest(BaseModel):
    message: str
    system: Optional[str] = None
    model: Optional[str] = None
    temperature: float = 0.7
    max_tokens: int = 1024

async def stream_ollama(request: ChatRequest) -> AsyncGenerator[bytes,
None]:
    model = request.model or settings.default_llm_model
    messages = []

```

```

        if request.system:
            messages.append({"role": "system", "content": request.system})
            messages.append({"role": "user", "content": request.message})

        payload = {
            "model": model,
            "messages": messages,
            "stream": True,
            "options": {
                "temperature": request.temperature,
                "num_predict": request.max_tokens,
            }
        }

        async with httpx.AsyncClient(timeout=120.0) as client:
            try:
                async with client.stream(
                    "POST",
                    f"{settings.ollama_host}/api/chat",
                    json=payload
                ) as response:
                    if response.status_code != 200:
                        error_text = await response.aread()
                        yield f"data: {json.dumps({'error': error_text.decode()})}\n\n".encode()
                        return

                    async for line in response.aiter_lines():
                        if not line.strip():
                            continue
                        try:
                            data = json.loads(line)
                            if content := data.get("message", {})).get("content"):
                                yield f"data: {json.dumps({'content': content})}\n\n".encode()
                            if data.get("done"):
                                yield f"data: {json.dumps({'done': True})}\n\n".encode()
                                break
                        except json.JSONDecodeError:
                            continue
                        except httpx.ConnectError:
                            yield f"data: {json.dumps({'error': 'Cannot connect to Ollama. Run: ollama serve'})}\n\n".encode()
                        except Exception as e:
                            yield f"data: {json.dumps({'error': str(e)})}\n\n".encode()

    @router.post("/stream")
    async def chat_stream(request: ChatRequest):
        return StreamingResponse(
            stream_ollama(request),
            media_type="text/event-stream",
            headers={
                "Cache-Control": "no-cache",
                "X-Accel-Buffering": "no",
            }
        )

```

```

@router.post("/")
async def chat(request: ChatRequest):
    model = request.model or settings.default_llm_model
    messages = []
    if request.system:
        messages.append({"role": "system", "content": request.system})
    messages.append({"role": "user", "content": request.message})

    async with httpx.AsyncClient(timeout=60.0) as client:
        try:
            response = await client.post(
                f"{settings.ollama_host}/api/chat",
                json={"model": model, "messages": messages, "stream":
False})
        except:
            if response.status_code != 200:
                raise HTTPException(status_code=502, detail="Ollama request
failed")
            data = response.json()
            return {"content": data.get("message", {}).get("content", "")}
        except httpx.ConnectError:
            raise HTTPException(status_code=503, detail="Ollama not
available")

```

Create `apps/ai-service/routers/analysis.py`:

python

```

from fastapi import APIRouter, HTTPException, BackgroundTasks
from pydantic import BaseModel
from typing import Optional, List
import asyncio
import logging
import subprocess
import json
import os
import tempfile

from config import settings

router = APIRouter()
logger = logging.getLogger(__name__)

class SilenceDetectionRequest(BaseModel):
    media_path: str
    threshold_db: float = -40.0
    min_silence_duration: float = 0.5
    min_content_duration: float = 0.3

class SilenceRegion(BaseModel):
    start: float
    end: float
    duration: float
    average_db: float

class TranscribeRequest(BaseModel):
    media_path: str
    model: str = "base"
    language: Optional[str] = None
    word_timestamps: bool = True

```

```

class TranscriptSegment(BaseModel):
    id: str
    start: float
    end: float
    text: str
    confidence: float
    words: Optional[List[dict]] = None

class SceneDetectionRequest(BaseModel):
    media_path: str
    threshold: float = 0.4

async def detect_silence_ffmpeg(media_path: str, threshold_db: float,
min_duration: float) -> List[SilenceRegion]:
    """Use FFmpeg silencedetect filter to find silent regions."""
    cmd = [
        "ffmpeg", "-i", media_path,
        "-af", f"silencedetect=noise={threshold_db}dB:d={min_duration}",
        "-f", "null", "-"
    ]

    proc = await asyncio.create_subprocess_exec(
        *cmd,
        stdout=asyncio.subprocess.PIPE,
        stderr=asyncio.subprocess.PIPE
    )
    , stderr = await proc.communicate()
    stderr_text = stderr.decode("utf-8", errors="replace")

    silences = []
    lines = stderr_text.split('\n')
    current_start = None

    for line in lines:
        if "silence_start:" in line:
            try:
                current_start = float(line.split("silence_start:")[-
1].strip())
            except ValueError:
                pass
            elif "silence_end:" in line and current_start is not None:
                try:
                    parts = line.split("|")
                    end = float(parts[0].split("silence_end:")[-1].strip())
                    duration = end - current_start
                    silences.append(SilenceRegion(
                        start=current_start,
                        end=end,
                        duration=duration,
                        average_db=threshold_db
                    ))
                    current_start = None
                except (ValueError, IndexError):
                    pass

    return silences

@router.post("/silence")
async def detect_silence(request: SceneDetectionRequest):

```

```

        if not os.path.exists(request.media_path):
            raise HTTPException(status_code=404, detail=f"File not found:
{request.media_path}")

    try:
        silences = await detect_silence_ffmpeg(
            request.media_path,
            request.threshold_db,
            request.min_silence_duration
        )
        return {
            "silences": [s.dict() for s in silences],
            "count": len(silences),
            "total_silence_duration": sum(s.duration for s in silences)
        }
    except Exception as e:
        logger.error(f"Silence detection failed: {e}")
        raise HTTPException(status_code=500, detail=str(e))

@router.post("/transcribe")
async def transcribe_media(request: TranscribeRequest):
    if not os.path.exists(request.media_path):
        raise HTTPException(status_code=404, detail=f"File not found:
{request.media_path}")

    try:
        import whisper
        model = whisper.load_model(request.model)
        options = {
            "word_timestamps": request.word_timestamps,
            "verbose": False,
        }
        if request.language:
            options["language"] = request.language

        result = model.transcribe(request.media_path, **options)

        segments = []
        for i, seg in enumerate(result.get("segments", [])):
            segment = {
                "id": str(i),
                "start": seg["start"],
                "end": seg["end"],
                "text": seg["text"].strip(),
                "confidence": seg.get("avg_logprob", 0.0),
            }
            if request.word_timestamps and "words" in seg:
                segment["words"] = [
                    {"word": w["word"], "start": w["start"], "end":
w["end"], "confidence": w.get("probability", 0.0)}
                    for w in seg["words"]
                ]
            segments.append(segment)

        return {
            "segments": segments,
            "full_text": result.get("text", ""),
            "language": result.get("language", "unknown"),
            "duration": result.get("duration", 0.0),

```

```

    ]
    except ImportError:
        raise HTTPException(
            status_code=503,
            detail="Whisper not installed. Run: pip install openai-whisper"
        )
    except Exception as e:
        logger.error(f"Transcription failed: {e}")
        raise HTTPException(status_code=500, detail=str(e))

@router.post("/scenes")
async def detect_scenes(request: SceneDetectionRequest):
    if not os.path.exists(request.media_path):
        raise HTTPException(status_code=404, detail=f"File not found: {request.media_path}")

    try:
        cmd = [
            "ffmpeg", "-i", request.media_path,
            "-vf", f"select='gt(scene,{request.threshold})',showinfo",
            "-vsync", "vfr",
            "-f", "null", "-"
        ]

        proc = await asyncio.create_subprocess_exec(
            *cmd,
            stdout=asyncio.subprocess.PIPE,
            stderr=asyncio.subprocess.PIPE
        )
        _, stderr = await proc.communicate()
        stderr_text = stderr.decode("utf-8", errors="replace")

        boundaries = []
        for line in stderr_text.split('\n'):
            if "pts time:" in line and "Parsed showinfo" in line:
                try:
                    pts_part = [p for p in line.split() if "pts time:" in p][0]
                    time = float(pts_part.split("pts time:")[1])
                    boundaries.append({"time": time, "score": request.threshold, "type": "cut"})
                except (ValueError, IndexError):
                    pass

        return {"scenes": boundaries, "count": len(boundaries)}
    except Exception as e:
        logger.error(f"Scene detection failed: {e}")
        raise HTTPException(status_code=500, detail=str(e))

@router.post("/embed")
async def embed_text(payload: dict):
    """Generate text embeddings for semantic search."""
    text = payload.get("text", "")
    try:
        from sentence_transformers import SentenceTransformer
        model = SentenceTransformer("all-MiniLM-L6-v2")
        embedding = model.encode(text).tolist()
        return {"embedding": embedding, "dimensions": len(embedding)}
    except ImportError:

```

```

        raise HTTPException(
            status_code=503,
            detail="sentence-transformers not installed"
        )

```

Create `apps/ai-service/routers/models.py`:

python

```

from fastapi import APIRouter, HTTPException
from pydantic import BaseModel
from typing import List, Optional
import httpx
import logging
from config import settings

router = APIRouter()
logger = logging.getLogger(__name__)

@router.get("/ollama")
async def list_ollama_models():
    async with httpx.AsyncClient(timeout=10.0) as client:
        try:
            response = await client.get(f"{settings.ollama_host}/api/tags")
            if response.status_code != 200:
                raise HTTPException(status_code=502, detail="Ollama request failed")
            data = response.json()
            return {"models": data.get("models", []), "status": "connected"}
        except httpx.ConnectError:
            return {"models": [], "status": "disconnected", "error": "Cannot connect to Ollama"}

class PullModelRequest(BaseModel):
    model: str

@router.post("/ollama/pull")
async def pull_ollama_model(request: PullModelRequest):
    async with httpx.AsyncClient(timeout=300.0) as client:
        try:
            response = await client.post(
                f"{settings.ollama_host}/api/pull",
                json={"name": request.model}
            )
            return {"status": "started", "model": request.model}
        except httpx.ConnectError:
            raise HTTPException(status_code=503, detail="Cannot connect to Ollama")

@router.get("/status")
async def models_status():
    ollama_ok = False
    try:
        async with httpx.AsyncClient(timeout=5.0) as client:
            resp = await client.get(f"{settings.ollama_host}/api/tags")
            ollama_ok = resp.status_code == 200
    except Exception:
        pass

    return {

```

```
"ollama": {"connected": ollama_ok, "host": settings.ollama_host},
"default_llm": settings.default_llm_model,
"default_whisper": settings.default_whisper_model,
"device": settings.device,
}
```

PHASE 1 — STARTUP SCRIPTS

Create `scripts/dev.sh`:

bash

```
#!/bin/bash
set -e

echo "🚀 Starting AXEW Development Environment..."

# Load environment
if [ -f .env ]; then
    export $(grep -v '^#' .env | xargs)
fi

# Start Rust backend in background
echo "🏗 Building & starting Rust backend..."
(cd crates/axew-core && cargo build 2>&1 | tail -5 && cargo run &)

# Start Python AI service in background
echo "🐍 Starting Python AI service..."
(cd apps/ai-service && \
    python3 -m venv .venv --quiet 2>/dev/null || true && \
    source .venv/bin/activate && \
    pip install -r requirements.txt --quiet 2>&1 | tail -3 && \
    python3 -m uvicorn main:app --host 127.0.0.1 --port 7002 --reload &)

# Give services time to start
sleep 3

# Start Electron frontend
echo "⚡ Starting Electron + React frontend..."
cd apps/desktop && pnpm dev:electron

echo "✅ AXEW is running!"
```

Create `scripts/install.sh`:

bash

```
#!/bin/bash
set -e

echo "📦 Installing AXEW dependencies..."

# Node packages
echo "📦 Installing Node packages..."
pnpm install

# Rust build
echo "🏗 Building Rust backend..."
cd crates/axew-core
```

```
cargo build
cd ../../

# Python AI service
echo "🔧 Setting up Python AI service..."
cd apps/ai-service
python3 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
cd ../../

echo ""
echo "✅ Installation complete!"
echo ""
echo "Optional: Install Ollama for AI features:"
echo "  https://ollama.com/download"
echo "  Then run: ollama pull llama3"
echo ""
echo "Run 'pnpm dev' to start the editor"
Make both scripts executable: chmod +x scripts/dev.sh scripts/install.sh
Add nanoid and path-browserify to the desktop app:
Run: cd apps/desktop && pnpm add nanoid path-browserify Run: cd
apps/desktop && pnpm add -D @types/path-browserify
```

PHASE 9 — PACKAGE MISSING DEPENDENCIES AND FIX TYPES

Run the following in `apps/desktop/`:

bash

```
pnpm add nanoid immer zustand framer-motion lucide-react clsx tailwind-merge @tanstack/react-query axios path-browserify
pnpm add -D @types/react @types/react-dom @types/path-browserify vite-plugin-electron vite-plugin-electron-renderer electron electron-builder concurrently wait-on
```

Create `apps/desktop/src/vite-env.d.ts`:

typescript

```
/// <reference types="vite/client" />
```

```
interface ImportMeta {
  readonly env: ImportMetaEnv
}
```

Ensure `nanoid` is declared. Add to `apps/desktop/src/lib/id.ts`:

typescript

```
export { nanoid } from 'nanoid'
```

PHASE 9 — FINAL INTEGRATION & BUILD VERIFICATION

Final Build Steps

Execute these commands in order:

1. **Install root dependencies:**

bash

```
pnpm install
```

2. **Install desktop deps:**

bash

```
cd apps/desktop && pnpm install
```

3. **Build shared types:**

bash

```
cd packages/shared-types && echo "Types package ready (TypeScript source, no build needed)"
```

4. **Build Rust backend:**

bash

```
cd crates/axew-core && cargo build
```

5. **Set up Python AI service:**

bash

```
cd apps/ai-service
python3 -m venv .venv
source .venv/bin/activate # On Windows: .venv\Scripts\activate
pip install fastapi uvicorn httpx pydantic pydantic-settings aiofiles
```

6. **Start development:**

bash

```
cd apps/desktop && pnpm dev:electron
```

ARCHITECTURE DOCUMENTATION

Create `docs/ARCHITECTURE.md`:

markdown

```
# AXEW Architecture
```

```
## Overview
```

```
AXEW is a monorepo-structured AI-native NLE built on three runtime layers:
```

```
### Layer 1: Frontend (Electron + React)
```

```
- `apps/desktop/` - Electron shell + React UI
```

```
- State: Zustand stores (project, timeline, playback, UI, AI, export)
- Styling: TailwindCSS with cinematic dark theme
- Animation: Framer Motion
- IPC: Electron contextBridge (secure, no nodeIntegration)
```

Layer 2: Backend (Rust)

```
- `crates/axew-core/` - Axum HTTP server on port 7001
- Media probing via ffmpeg
- Thumbnail generation via ffmpeg
- Export pipeline via ffmpeg
- SQLite caching via tokio-rusqlite
```

Layer 3: AI Service (Python)

```
- `apps/ai-service/` - FastAPI on port 7002
- Streaming chat via Ollama
- Silence detection via FFmpeg silencedetect filter
- Transcription via OpenAI Whisper
- Scene detection via FFmpeg scene filter
- Embeddings via sentence-transformers
```

Data Flow

User Action → Zustand Store → React Component ↓ Rust API (port 7001)
AI Service (port 7002) Ollama (port 11434)

Timeline Data Model

Non-destructive editing: clips reference source media by time range.
Timeline → Tracks → Clips → (mediaId, startTime, duration, inPoint, outPoint)

AI Integration Points

1. Chat: Direct Ollama streaming via AI service proxy
2. Silence Detection: FFmpeg-based, results used to create clips
3. Transcription: Whisper model generates segments mapped to timeline
4. Scene Detection: FFmpeg scene filter creates marker points
5. Future: Embedding search, LoRA fine-tuning, agent orchestration

Future GPU Acceleration

```
- CUDA kernels: frame decoding, thumbnail generation
- Vulkan compute: real-time effects pipeline
- TensorRT: inference acceleration for vision models
- Metal (macOS): hardware-accelerated video playback
```

PHASE 10 — FINAL CHECKS AND AUTOFIX

Cursor must now:

1. Scan all TypeScript files for import errors. Fix any missing type imports. Add `// @ts-ignore` only as absolute last resort.
2. Verify `apps/desktop/src/stores/timelineStore.ts` — the `setTimeline` function uses `useProjectStore.setState` with immer.

Confirm this works by checking Zustand version. If not, refactor to use `useProjectStore.getState().currentProject` mutation pattern.

3. Ensure `nanoid` is imported everywhere it is used. Add `import { nanoid } from 'nanoid'` at the top of any file using it.
4. Ensure all Radix UI component imports in any dialog or modal components are correct.
5. Add `"pydantic-settings"` to `requirements.txt` if not already present.
6. Verify `electron/main.ts` references `preload.js` (not `.ts`) in the `preload` path since it points to compiled output.
7. Ensure `apps/desktop/electron/electron-env.d.ts` is included in TypeScript compilation.
8. Add the `lazy_static` dependency to `crates/axew-core/Cargo.toml` and run `cargo build` to verify compilation.
9. If Rust build fails due to `lazy_static`, replace with `std::sync::OnceLock` or `tokio::sync::RwLock` and restructure export job storage accordingly.
10. Run `cd apps/desktop && pnpm type-check` and fix any type errors